Transportation Research Part E 202 (2025) 104297

Contents lists available at ScienceDirect

Transportation Research Part E

journal homepage: www.elsevier.com/locate/tre

Inventory routing problem of automotive parts considering time-varying demands: A machine learning enhanced branch-and-price approach

Yu Wang [a,*], Renrong Zheng [b], Chengji Liang [a], Jian Shi [c]

[a] Logistics Research Center, Institute of Logistics Science and Engineering, Shanghai Maritime University, Shanghai, 201306, China
[b] School of Transportation, Key Laboratory of Road and Traffic Engineering of Ministry of Education, Tongji University, Shanghai, 201804, China
[c] Department of Engineering Technology and Department of Electrical and Computer Engineering, University of Houston, TX, 77004, USA

ARTICLE INFO

Keywords:
Automotive inbound logistics
Inventory routing
Time-varying demands
Branch-and-price
Machine learning

ABSTRACT

With the rise of mass customization and smart manufacturing, the automotive industry is rapidly transitioning to improve responsiveness, manage highly diversified customer orders, and reduce inbound logistics costs. To address this challenge, this paper proposes a new variant of the multi-period inventory routing problem, which focuses on coordinating discrete, time-varying demands for auto parts on the assembly line with predetermined packages at suppliers over a finite short-term time horizon (e.g., on an hourly basis). The objective is to minimize the total transportation and inventory cost by making aperiodic decisions on collection quantities and traveling routes simultaneously for an inbound warehouse near the assembly plant. An integer programming (IP) formulation with time-indexed variables is tailored for the problem to analyze the feasibility conditions. Then, a reformulation is designed to make the problem more tractable, based on which a novel machine learning enhanced branch-and-price algorithm (BPL) is proposed, where prediction-based cuts are embedded to accelerate the pricing procedure. Experiments on real-scale instances demonstrate that the algorithm consistently achieves near-optimal solutions, with a gap of 4.42% on average from the best-found lower bound, and reduces computation time by over 90% compared to directly solving the IP model by CPLEX. The proposed learning technique is computationally efficient, capable of shortening the total calculation time by an average of 13%. This work facilitates timely decision-making and offers new insights into multi-period inventory routing for inbound logistics.

1. Introduction

1.1. Background and motivation

With the demand for mass customization and smart manufacturing in the automotive industry, manufacturers are looking for more timely and flexible operational schemes for the inbound logistics of auto parts. Compared with other supply chain logistics problems, automotive inbound logistics is particularly complex due to the need to efficiently deliver thousands of part types from various suppliers to a mix-model assembly line. The demands for auto parts are time-varying, driven by the assembly line's

* Corresponding author.
E-mail addresses: wangyu@shmtu.edu.cn (Y. Wang), renrongzheng@tongji.edu.cn (R. Zheng), liangcj@shmtu.edu.cn (C. Liang), jshi14@uh.edu (J. Shi).

https://doi.org/10.1016/j.tre.2025.104297
Received 8 March 2024; Received in revised form 21 June 2025; Accepted 6 July 2025
Available online 21 July 2025
1366-5545/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

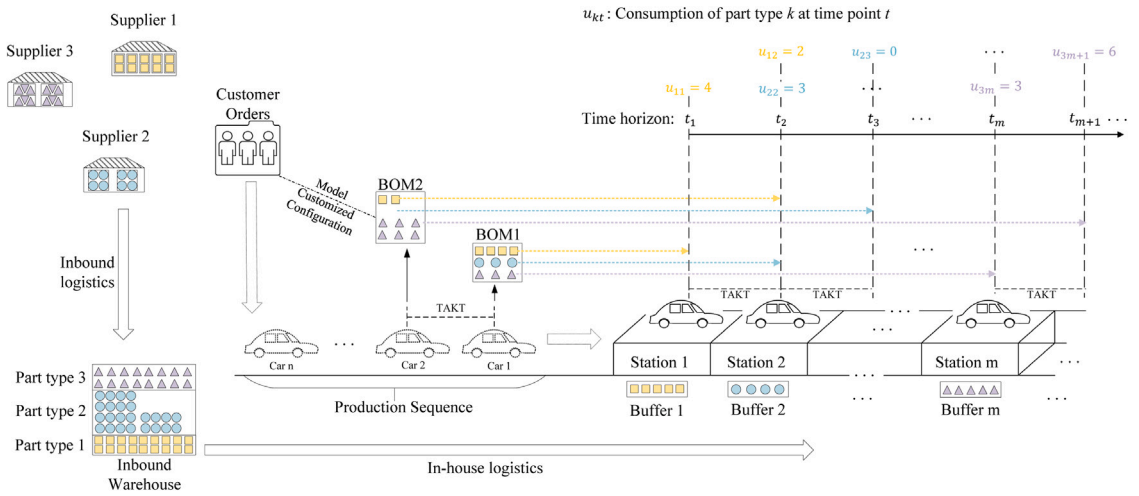


Fig. 1. An example of the time-varying demands from the assembly line.

requirements. For example, in a typical assembly line of SAIC Motor Corp., Ltd. we studied, parts are required every 1.5 min at 120 workstations to meet the needs of six basic car models and various configurations. This critical demand makes the traditional inbound logistics operations, where parts are delivered in predetermined quantities and stable frequencies based on forecasted customer demands and long-term production plans, challenging. Issues including high inventory costs, sudden shortages of parts, and dead stock could arise and lead to serious loss. These issues are exacerbated by increased production flexibility, diversified customer orders, and the pursuit of Just-in-Time (JIT) auto-part supply, making the reliable, efficient and economical delivery of auto parts a key concern for global automotive supply chains.

To address this crucial challenge, this paper considers a multi-period inventory routing problem (IRP) for an inbound warehouse to achieve timely and efficient delivery of packed auto parts based on time-varying demands, with the goal of improving services for customized and intelligent automobile productions and further reducing the warehousing and transportation costs of auto parts. The traditional IRP focuses on developing a periodic delivery strategy by deciding customer visits, delivery quantities, and travel routes, aiming to minimize transportation costs under inventory and routing constraints while avoiding stockouts. Motivated by the practical operations of automotive manufacturers, we introduce a new variant of the IRP, focused on flexible, aperiodic distribution decisions, coordinating original package quantities of various part types and their time-varying demands over a short-term horizon. This aims to achieve cost-efficiency in transportation and inventory management while keeping stock of auto parts at secure and economical levels.

The time-varying demands of auto parts are generated from highly diversified customer orders on the automotive mix-model assembly line. On a typical automotive assembly line, every vehicle model has a distinct Bill of Materials (BOM) outlining its component requirements. As partially assembled vehicles move along the line, aligned with specific customer orders, each station retrieves necessary parts from its buffer in accordance with the BOM. The assembly operates at a fixed rhythm, known as TAKT time, and the demand for each type of part at a given moment depends on the specific BOM associated with the vehicle at that station. Moreover, the parts of each type are packaged in uniform quantities and sizes by suppliers and are delivered to an inbound warehouse to replenish the buffers. To avoid excess or shortages, the delivery of parts must align with time-varying demands, considering that only integer multiples of the original packaging quantities can be shipped.

Fig. 1 illustrates an example of predetermined packages of auto parts and the time-varying demands generated by customer orders on the assembly line. Three types of parts (type 1, 2, and 3) are depicted as squares, circles, and triangles in the figure, respectively, each packaged in predetermined quantities (2, 4, and 3 parts per package, respectively) by Suppliers 1, 2, and 3. As semi-finished cars enter the assembly line, each corresponding to a specific customer order, their BOMs define the required parts, and the required quantities change over time based on the sequence of assembly and the TAKT. For example, Car 1 requires 4 units of part type 1 at t_1, 3 units of part type 2 at $t_2 = t_1$+TAKT, and 3 units of part type 3 at $t_m = t_1 + (m-1)$TAKT, while the following Car 2 requires 2 units of part type 1 at t_2, and 6 units of part type 3 at $t_{m+1} = t_1 + m$TAKT. Since the production schedule is usually finalized 1 to 2 h in advance, the demand for each part type at every TAKT point within a finite planning horizon can be determined based on the assembly sequence. These scheduled part needs are considered as the time-varying demands for inbound warehouse replenishment.

As shown in Fig. 1, for each part type, the required quantities vary over the horizon according to diverse customer orders and assembly sequence. Besides, the quantity of each part type required at each TAKT differs from each other and is usually smaller than the quantity in the corresponding package. It is necessary to coordinate the delivery packages of every part type to the time-varying demands to avoid dead stock or suddenly shortage.

1.2. Our contribution

This paper addresses an inventory routing problem of delivering packed auto parts to an inbound warehouse, motivated by the real-world operations of an automotive manufacturer in Shanghai, China. A homogeneous fleet collects various types of auto parts from scattered suppliers and delivers them to the warehouse, with routing and inventory decisions made simultaneously to minimize total transportation and inventory costs. While many variants of IRPs have been studied, typically focusing on different characteristics of shipping times, planning horizon, distribution plan structure, and the objective of the plan, none of the existing research considers the coordination of package quantities with the discrete time-varying demands of the automotive assembly process over a finite short-term time horizon, where shipments spanning multiple time periods are permitted. The contributions of this paper are:

- A novel variant of the multi-period inventory routing problem is proposed to generate more accurate and flexible operational decisions for automotive inbound logistics by coordinating pre-packed auto parts to their time-varying demands on an hourly basis. It will contribute to dealing with highly diversified customer orders and complying with the flexibility of intelligent production.
- The problem's feasibility is examined based on a time-indexed integer programming model. To improve computational tractability, a reformulation approach is presented that transforms the model into a set partitioning type of model.
- An exact branch-and-price (BnP) framework is developed to solve the problem efficiently, in which a machine learning-based method, a hybrid branching strategy and early termination rules are developed to accelerate the solution process. The algorithm, as well as the machine learning technique, is proven to be computationally efficient and effective in improving decision-making in the practice of automotive inbound logistics by obtaining near-optimal solutions.

Although our approach is derived from automotive inbound logistics, it can be easily extended to other inbound logistics operations of similar manufacturing companies with assembly lines. For example, similar inventory routing problems are also commonly seen in pharmaceutical and electronic industries, which involve the mix-model assembly line and diverse types of parts that need to be delivered in different packages from various suppliers.

The remaining structure of the paper is arranged as follows: Section 2 reviews relevant literature. Section 3 describes the problem in detail, presents an IP formulation, and analyzes feasibility conditions. In Section 4, we develop a machine learning enhanced branch-and-price algorithm (BPL) to efficiently address the problem. Section 5 reports the results of computational experiments using synthetic data. Finally, Section 6 summarizes the main findings and concludes the study.

2. Literature review

This section reviews two streams of related research: the first focuses on inbound logistics practices for auto parts, and the second addresses inventory routing problem formulations and solutions.

2.1. Inbound logistics of auto parts

The inbound logistics of auto parts is closely tied to production strategies, which have evolved from stock-push to demand-pull paradigms over the past few decades (Duguay et al., 1997; Harrison, 2001). This shift, driven by just-in-time (JIT) and lean manufacturing, has increased the complexity of the inbound transportation systems, which requires them to be more flexible and synchronized with production demands.

Most early studies in this domain focus on routing decisions under simplified assumptions. For example, Chuah and Yingling (2005) proposed an algorithm combining column generation and tabu search for fixed-route planning, while Jiang et al. (2010) and Hosseini et al. (2014) applied metaheuristics to similar problems. These works assume stable demand patterns and fixed delivery schedules. As outlined by Boysen et al. (2015), inbound logistics in the automotive industry typically employs three transportation strategies: point-to-point, milk-run, and cross-docking. This review primarily focuses on milk-run transportation problems, since our problem involves routing decisions among multiple suppliers for each shipment.

Subsequent studies introduced more practical constraints, such as time windows and delivery flexibility. For example, Ohlmann et al. (2008) incorporated time windows and demands split, and a tabu search heuristic and a binary integer program were developed to solve the problem. Lin et al. (2015) combined greedy and tabu search in a two-stage heuristic to address problem complexity. These studies, however, still largely separate inventory from routing considerations.

A recent trend is the integration of more nuanced logistics settings, such as drop-and-pull operations (Chen et al., 2021) and flexible pickup volumes (Jia and Chen, 2023), demonstrating a move toward models that more closely mirror real-world inbound operations. Baller et al. (2022) introduced cyclic inventory constraints and valid inequalities, advancing model scalability and solution robustness that is tailored to an automotive factory.

While most of the above studies assume fixed or aggregated demand, a smaller stream explores coordination between inbound logistics and dynamic production needs. Mao et al. (2020) linked vehicle routing to assembly-line operations, minimizing joint production and logistics costs. Wang and Chen (2019) and Chitsaz et al. (2020) incorporated TAKT time-based demand fluctuations into scheduling and planning models. Baals et al. (2023) considered a JIT routing problem for OEMs with time window constraints, and developed a metaheuristic combining Large Neighborhood Search (LNS) and Local Search (LS) to minimize earliness-tardiness costs. These studies demonstrate that aligning logistics with real-time production requirements offers potential for greater efficiency.

Table 1
Literatures on inbound logistics operations of auto parts.

Reference	Inventory decision	Routing decision	Multi periods	Time-varying demands	Packed parts	Method
Chuah and Yingling (2005)	×	✓	×	×	×	CG & TS
Jiang et al. (2010)	×	✓	×	×	×	TS
Hosseini et al. (2014)	×	✓	×	×	×	HS
Ohlmann et al. (2008)	×	✓	×	×	×	TS
Lin et al. (2015)	×	✓	×	×	×	GM & TS
Chen et al. (2021)	×	✓	×	×	×	GA
Baller et al. (2022)	×	×	✓	✓	×	MILP
Jia and Chen (2023)	×	✓	×	×	×	BnP
Mao et al. (2020)	×	✓	×	×	×	GA
Wang and Chen (2019)	×	✓	✓	✓	✓	CG
Chitsaz et al. (2020)	✓	✓	✓	✓	×	BnC
Baals et al. (2023)	×	✓	×	×	×	LNS & LS
This paper	✓	✓	✓	✓	✓	Learning based BnP

CG: column generation; TS: tabu search; HS: harmony search; MILP: mixed-integer linear programming; GM: greedy method; GA: genetic algorithm; BnP: branch-and-cut; LNS: large neighborhood search; LS: local search; BnP: branch-and-price.

Table 1 highlights the key differences between the literature on auto parts inbound logistics and the proposed approach. We can observe that most of the related literature considers relatively long-term transportation planning without inventory features or practical characteristics such as time-varying demands and original packages of parts. This limitation would influence the decisions on the loading and routing of daily shipments, potentially resulting in higher inventory and transportation expenses in real-world scenarios. The high diversity of customer orders and the development of intelligent assembly lines are driving the automotive industry to seek more flexible and timely solutions for auto parts delivery. Balancing inventory holding costs with transportation expenses remains a central challenge. This paper considers the coordination of the time-varying demands from the assembly line and the original packages at the suppliers for the auto parts aligning with the current trend of lean and smart production requirements in the automotive industry.

2.2. Inventory routing problems

Our problem involves both inventory and routing decisions. Traditional IRPs, originating from the work of Bell et al. (1983), are closely linked to classical vehicle routing problems (VRPs) that incorporate inventory considerations. These models primarily emphasize routing strategies, as highlighted by Konstantakopoulos et al. (2022). Computational studies have shown that the average saving of integrating transportation and inventory management is around 24% (Archetti and Speranza, 2016). IRP is an extremely complex problem itself since three decisions need to be made in general: when to visit the customers, how many items to be delivered each time, and what are the sequences of the vehicles visiting related customers. Numerous IRP variants have been studied in the past 40 years, and classification features such as time horizon, network structure, inventory policy, and fleet size, are studied by Coelho et al. (2014).

According to the survey by Zajac and Huber (2021), current IRP problems all focus on the tactical level over the mid-term planning horizon, which is mostly considered cyclic replenishment and solved by heuristics. For example, Rau et al. (2018) addressed a cyclic green inventory management problem with multiple objectives and developed a discrete multi-swarm particle swarm optimization approach. Kheiri (2020) introduced an enhanced sequence-based selection hyper-heuristic tailored for the Air Liquide inventory-routing context. Mahjoob et al. (2021) explored an inventory routing scenario in the flour industry, incorporating time windows and split deliveries, and proposed an effective solution based on an a priori method.

More recently, Archetti and Ljubić (2022) conducted a comparative study between aggregated and disaggregated formulations of the IRPs, along with the benchmarked advanced exact solution techniques. They noted that most exact approaches for IRPs rely heavily on the branch-and-cut framework, with limited research exploring pricing mechanisms. Desaulniers et al. (2016) introduced a novel formulation and a branch-price-and-cut method enhanced with valid inequalities. Contemporary research has expanded to include objectives and constraints such as customer time windows, demand uncertainty, and continuous-time dynamics. For example, Tiniç et al. (2021) applied a branch-price-and-cut method to IRPs with time windows, evaluating cases of both static and dynamic customer demand. Coelho et al. (2020) addressed a multi-depot, multi-product IRP with a heterogeneous fleet, proposing a variable MIP neighborhood descent heuristic. Liu et al. (2021) developed a location-inventory-routing framework for perishables, incorporating carbon emissions and freshness decay. Skálnes et al. (2022) handled time-dependent demands using a strengthened formulation and branch-and-cut algorithm. Product substitution was integrated into IRPs by Mahmutoğulları and Yaman (2023), who also devised a new family of valid inequalities for a tailored branch-and-cut method. Additionally, Cavero et al. (2024) tackled short-sea oil transportation by assigning different oil types to vessels across multiple ports, using a neighborhood-search-based matheuristic. Diabat et al. (2024) explored zero-inventory-ordering IRPs, allowing customer replenishment only at zero stock levels, and compared this policy to traditional approaches via a branch-and-cut scheme.

Table 2
Literature on inventory routing problems.

Reference	Multi-product	Time-varying demands	Deliver in packages	Cross-period shipments	Method
Rau et al. (2018)	×	×	×	×	MSPSO
Kheiri (2020)	×	✓	×	✓	Hyper-heuristics
Mahjoob et al. (2021)	×	✓	×	×	Priori-method
Desaulniers et al. (2016)	×	✓	×	×	BPnC
Tiniç et al. (2021)	×	✓	×	×	BPnC
Coelho et al. (2020)	✓	✓	×	×	Variable MIP ND
Liu et al. (2021)	✓	✓	×	×	YALMIP toolbox
Skálnes et al. (2022)	×	✓	×	×	BnC
Mahmutoğulları and Yaman (2023)	✓	✓	×	×	BnC
Cavero et al. (2024)	×	✓	×	×	GRASP-VND
Diabat et al. (2024)	×	✓	×	×	BnC
This paper	✓	✓	✓	✓	Learning based BnP

MSPSO: multi-swarm particle swarm optimization; BPnC: branch-and-price-and-cut; ND:neighborhood descent; BnC: branch-and-cut; AL NS: adaptive large neighborhood search; BnP: branch-and-price; GRASP-VND: greedy randomized adaptive search procedure with variable neighborhood descent.

Table 2 shows the key differences between the IRP literature and the proposed problem. While previous literature mostly considered a distribution scenario with one or several depots offering identical products to customers with continuously consuming demands, they usually tend to generate constant plans within each period. None of them considered the inbound logistics of the automotive industry, where multi-type auto parts in the warehouse are replenished based on discrete time-varying demands on an hourly basis deriving from diverse customer orders. These demands have to be carefully coordinated with the predetermined packages of the suppliers to ensure flexible and effective deliveries. Therefore, the mathematical model and related algorithms in existing IRP studies cannot be applied directly to our problem.

This paper introduces a branch-and-price algorithm (BPL) integrated with machine learning techniques. For foundational knowledge on column generation and the branch-and-price framework, we refer interested readers to Oğuz (2002) and Yu et al. (2019). While machine learning has recently been explored to enhance traditional exact methods (Václavík et al., 2018; Morabit et al., 2021; Yang et al., 2022), prior studies have largely focused on simplified scenarios. To the best of our knowledge, our approach is the first to incorporate online upper bound prediction across multiple subproblems within a complex, real-world automotive context.

In summary, existing literature on automotive inbound logistics primarily focuses on transportation with a uniform consumption rate of each auto part, while research on general IRP problems often addresses the distribution of identical products over a relatively long-term planning horizon without shipments spanning multiple periods. However, in the context of lean logistics and customization in the modern automotive industry, time-varying discrete demands over short-term planning periods and their coordination with suppliers' original packaging quantities must be considered. This is necessary to meet the actual consumption needs at each TAKT on the assembly line and reduce logistics costs. Specifically, compared to traditional IRP literature, considering time-varying demands and packed auto parts during transportation over a short-term horizon introduces additional complexity to our problem and model. The short-term horizon allows each shipment to span multiple periods, rather than being confined to a single period as in most studies. The various loading combinations of multi-type auto parts will generate multiple routing alternatives, and the delivery of auto parts in packages would lead to sharp fluctuations in inventory levels, while roughly meeting the discrete time-varying demands.

3. Problem formulation and analysis

We start by describing the problem in detail (Section 3.1), then present its integer programming formulation (IP) (Section 3.2), and finally analyze the feasibility conditions (Section 3.3).

3.1. Problem description

The inventory routing problem under consideration involves managing pre-packed auto parts with time-dependent demands. Specifically, a group of N suppliers, represented by the set $\mathbb{N}' = 1, 2, \ldots, N$, supplies K types of auto parts, defined by the set $\mathbb{K} = 1, 2, \ldots, K$, to an automotive assembly facility. These parts are stored at an inbound warehouse adjacent to the production line. The complete set of locations, including the warehouse, is denoted as $\mathbb{N} = 0 \cup \mathbb{N}'$, where node 0 represents the warehouse. Without loss of generality, each supplier $i \in \mathbb{N}'$ may supply multiple part types. For every $k \in \mathbb{K}$, parts are packed into standard, identical packages and delivered by the assigned supplier. The volume and unit quantity per package for part type k are denoted by b_k and h_k, respectively. Inventory levels for each part type at the warehouse are constrained within a given range, with u_k and l_k as the upper and lower bounds. In our problem, the production activities of suppliers are not considered. Namely, it is assumed that all types of parts are always sufficiently stocked at the suppliers. The planning horizon spans T discrete time periods (e.g., hourly intervals), expressed as $\mathbb{T} = 1, 2, \ldots, T$. The time-varying demand of each part type k at time t is denoted by δ_{kt}, and the warehouse initially holds p_{k1} units of each part type k.

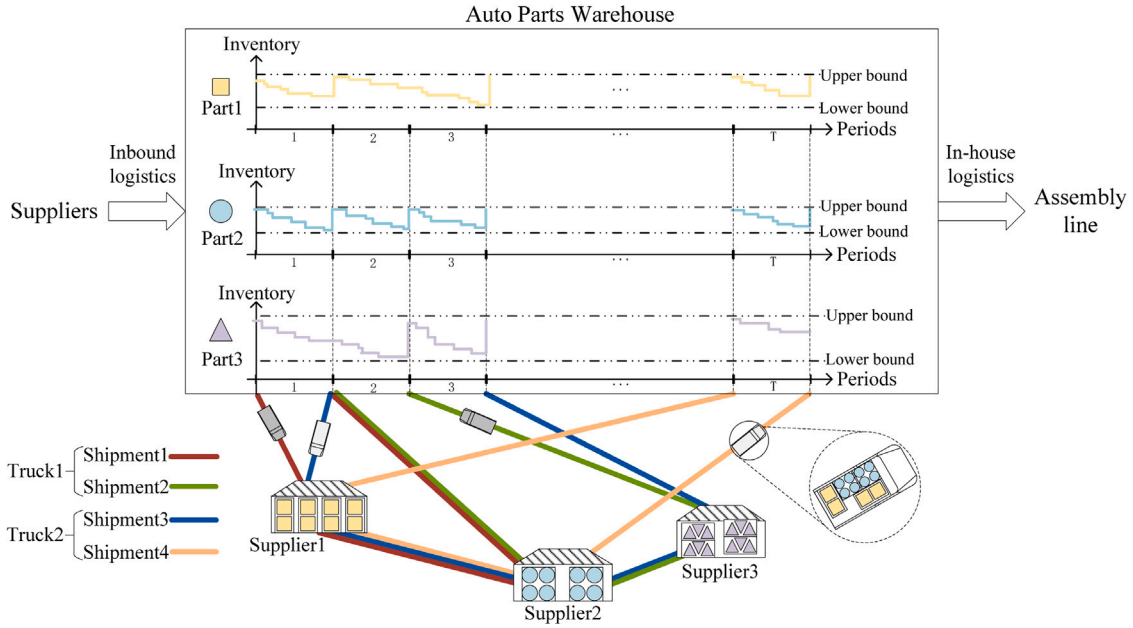


Fig. 2. A brief example of the proposed problem.

Given the known locations of the warehouse and suppliers, the travel time between any two nodes i and j in $\mathbb{N}$ (with $i \neq j$) is represented by ϵ_{ij}. A fixed handling time o is also assumed at each stop. The objective is to plan shipment schedules using a fleet of capacitated trucks over the planning horizon such that the inventory for all part types remains within the defined bounds, while minimizing the total cost of transportation and inventory.

Fig. 2 illustrates an example of the decision-making in the proposed inventory routing problem. Three types of parts (represented as squares, circles, and triangles) are supplied by three geographically dispersed suppliers. The required quantity for each part type per period is derived from the time-varying demands in the corresponding BOMs, as shown in Fig. 1. Each part type is packed in uniform packages, with packing quantities of 2, 4, and 3 units for parts 1, 2, and 3, respectively. In this example, two trucks are used to collect and deliver the parts, each performing two shipments: truck 1 handles Shipments 1 and 2, while truck 2 handles Shipments 3 and 4. As shown, shipments by the same truck cannot overlap in working time, and each shipment must comply with truck capacity and milk-run routing constraints. Since the parts are delivered in predetermined packages, as illustrated in the detailed view of Shipment 4, each shipment can only transport integer multiples of the package quantities for each part type. Consequently, it is necessary to determine the number of packages, as well as the timing and routing of each delivery, to ensure that the inventory levels of all part types remain within the specified limits and are appropriately replenished upon the completion of each shipment.

3.2. Integer programming formulation

We begin by introducing the sets, parameters, and decision variables for the proposed IRP variant of packed auto parts under time-varying demands.

Sets and Indexes:

$\mathbb{N}$	Set of nodes, $\mathbb{N} = \{0, 1, 2, \dots, N\}$, where 0 indicates the warehouse
$\mathbb{N}'$	Set of suppliers, $\mathbb{N}' = \mathbb{N} \backslash \{0\}$.
$\mathbb{E}$	Set of edges, $\mathbb{E} = \{(i, j) : i, j \in \mathbb{N}, i \neq j\}$.
$\mathbb{K}$	Part types set, $\mathbb{K} = \{1, 2, \dots, K\}$.
$\mathbb{V}$	Shipments set, $\mathbb{V} = \{1, 2, \dots, V\}$.
$\mathbb{T}$	Set of finite planning horizon, $\mathbb{T} = \{1, 2, \dots, T\}$, and $t \in \mathbb{T}$ indicates the starting moment of the period.

Parameters:

C_1	Inventory cost per unit.
C_2	Unit transportation cost.
b_k	The pre-package volume for part type $k \in \mathbb{K}$.
h_k	The predetermined number of parts per package for type $k \in \mathbb{K}$.
B	Capacity of trucks.
u_k	Inventory upper bound of each part type $k \in \mathbb{K}$ in the warehouse.
l_k	Inventory lower bound (safety stock) of each part type $k \in \mathbb{K}$ in the warehouse.
p_{k1}	Initial level of inventory of type $k \in \mathbb{K}$ at the outset of the planning horizon..
δ_{kt}	Demand quantity of part $k \in \mathbb{K}$ during period $t \in \mathbb{T}$.
r_{ki}	$r_{ki} = 1$ if supplier $i \in \mathbb{N}'$ provides part of type $k \in \mathbb{K}$, otherwise, $r_{ki} = 0$.
ε_{ij}	Traveling time on edge $(i, j) \in \mathbb{E}$.
o	Average handling time of visiting each node.
G	Number of capacitated trucks.
M	A sufficiently large constant.

Decision variables:

x_{ijv}	$x_{ijv} = 1$ if shipment $v \in \mathbb{V}$ travels from i to j directly, $(i, j) \in \mathbb{E}$, otherwise, $x_{ijv} = 0$.
y_{iv}	$y_{iv} = 1$ if shipment $v \in \mathbb{V}$ visit node $i \in \mathbb{N}$, otherwise, $y_{iv} = 0$.
β_{vkt}	$\beta_{vkt} \geq 0$denotes the integer quantity of packages for part $k \in \mathbb{K}$ sent at $t \in \mathbb{T}$ through shipment $v \in \mathbb{V}$.
s_{vt}	A binary variable equal to 1 if shipment $v \in \mathbb{V}$ is initiated at $t \in \mathbb{T}$, and 0 otherwise.
f_{vt}	A binary variable equal to 1 if shipment $v \in \mathbb{V}$ concludes at $t \in \mathbb{T}$, and 0 otherwise.
θ_{vt}	$\theta_{vt} = 1$ if point $t \in \mathbb{T}$ is larger than the starting point of shipment $v \in \mathbb{V}$, and is smaller than or equal to its finishing point, otherwise, $\theta_{vt} = 0$.
p_{kt}	A non-negative integer representing the inventory of type $k \in \mathbb{K}$ at the start of $t \in \mathbb{T}$.
e_{kt}	A non-negative integer representing the inventory type $k \in \mathbb{K}$ at the end of $t \in \mathbb{T}$.
σ_{iv}	The sequence of shipment $v \in \mathbb{V}$ visiting node $i \in \mathbb{N}'$.

Based on the above notations, the proposed IRP variant is formulated as the integer linear programming model (IP) below.

$$\text{(IP)} \quad \min C_1 \sum_{t=1}^{T} \sum_{k=1}^{K} \frac{p_{kt} + e_{kt}}{2} + C_2 \sum_{v=1}^{V} \sum_{(i,j) \in \mathbb{E}} \varepsilon_{ij} x_{ijv} \tag{1}$$

$$\text{s.t. } p_{kt} = p_{k1} - \sum_{z=1}^{t-1} \delta_{kz} + \sum_{v=1}^{V} \sum_{z=1}^{t} \beta_{vkt} \cdot h_k, \forall t \in \mathbb{T} \backslash \{1\}, k \in \mathbb{K} \tag{2}$$

$$e_{kt} = p_{kt} - \delta_{kt}, \forall t \in \mathbb{T}, k \in \mathbb{K} \tag{3}$$

$$p_{kt} \geq l_k, \forall t \in \mathbb{T}, k \in \mathbb{K} \tag{4}$$

$$p_{kt} \leq u_k, \forall t \in \mathbb{T}, k \in \mathbb{K} \tag{5}$$

$$\sum_{t=1}^{T} s_{vt} \leq 1, \forall v \in \mathbb{V} \tag{6}$$

$$\sum_{t=1}^{T} f_{vt} \leq \sum_{t=1}^{T} s_{vt}, \forall v \in \mathbb{V} \tag{7}$$

$$\sum_{t=1}^{T} f_{vt} \cdot t - \sum_{t=1}^{T} s_{vt} \cdot t \geq \sum_{(i,j) \in \mathbb{E}} \varepsilon_{ij} \cdot x_{ijv} + \sum_{i=0}^{N} y_{iv} \cdot o, \forall v \in \mathbb{V} \tag{8}$$

$$y_{iv} \leq \sum_{k=1}^{K} \sum_{t=1}^{T} \beta_{vkt} \cdot r_{ki}, \forall i \in \mathbb{I}', v \in \mathbb{V} \tag{9}$$

$$M \cdot y_{iv} \geq \sum_{k=1}^{K} \sum_{t=1}^{T} \beta_{vkt} \cdot r_{ki}, \forall i \in \mathbb{I}', v \in \mathbb{V} \tag{10}$$

$$M \cdot y_{0v} \geq \sum_{i=1}^{N} y_{iv}, \forall v \in \mathbb{V} \tag{11}$$

$$\sum_{k=1}^{K} \beta_{vkt} \geq f_{vt}, \forall v \in \mathbb{V}, t \in \mathbb{T} \tag{12}$$

$$\beta_{vkt} \le M \cdot f_{vt}, \forall v \in \mathbb{V}, k \in \mathbb{K}, t \in \mathbb{T} \tag{13}$$

$$\sum_{k=1}^{K}\sum_{t=1}^{T} \beta_{vkt} \cdot b_k \le B, \forall v \in \mathbb{V} \tag{14}$$

$$\sum_{j=0, j\neq i}^{N} x_{ijv} = y_{iv}, \forall v \in \mathbb{V}, i \in \mathbb{N} \tag{15}$$

$$\sum_{j=0, j\neq i}^{N} x_{ijv} - \sum_{j=0, j\neq i}^{N} x_{jiv} = 0, \forall v \in \mathbb{V}, i \in \mathbb{N} \tag{16}$$

$$\sigma_{iv} - \sigma_{jv} + N \cdot x_{ijv} \le N - 1, \forall v \in \mathbb{V}, i, j \in \mathbb{I}', i \neq j \tag{17}$$

$$\theta_{vt} \cdot t \le \sum_{z=1}^{T} f_{vz} \cdot z, \forall v \in \mathbb{V}, t \in \mathbb{T} \tag{18}$$

$$(1 - \theta_{vt}) \cdot M + \theta_{vt} \cdot t \ge \sum_{z=1}^{T} s_{vz} \cdot z + 1, \forall v \in \mathbb{V}, t \in \mathbb{T} \tag{19}$$

$$\sum_{t=1}^{T} \theta_{vt} \ge \sum_{t=1}^{T} f_{vt} \cdot t - \sum_{t=1}^{T} s_{vt} \cdot t, \forall v \in \mathbb{V} \tag{20}$$

$$G \ge \sum_{v=1}^{V} \theta_{vt}, \forall t \in \mathbb{T} \tag{21}$$

$$x_{ijv}, y_{iv}, s_{vt}, f_{vt}, \theta_{ct} \in \{0, 1\}, \beta_{vkt}, p_{kt}, e_{kt}, \sigma_{bi} \in Z^{+} \tag{22}$$

The objective function (1) seeks to minimize the combined costs of inventory and transportation. The inventory cost, given by $C_1\sum_{t=1}^{T}\sum_{k=1}^{K}\frac{p_{kt}+e_{kt}}{2}$, is based on the average inventory level of each part $k \in \mathbb{K}$ over each period $t \in \mathbb{T}$. The transportation cost, $C_2\sum_{v=1}^{V}\sum_{(i,j)\in\mathbb{E}}\varepsilon_{ij}x_{ijv}$, depends on the routing decisions of shipments $v \in \mathbb{V}$. The model's constraints are categorized into the following five groups:

(i) Constraints on the inventory of each part type: Constraint (2) specifies the beginning inventory for each part $k \in \mathbb{K}$ in period $t \in \mathbb{T}\backslash 1$, calculated as its initial stock p_{k1} minus the total consumption before t, plus the accumulated delivered quantity from earlier periods and at the start of period t. Likewise, constraint (3) determines the ending inventory of part $k \in \mathbb{K}$ in period t, as the beginning inventory minus the consumption during that period. Constraints (4) and (5) ensure that the inventory of part type $k \in \mathbb{K}$ in every period $t \in \mathbb{T}$ remains between predefined lower and upper limits.

(ii) Constraints on the working time for each shipment: Constraints (6) and (7) ensure that only one starting point and one finishing point can be selected for any shipment $v \in \mathbb{V}$. Constraint (8) guarantees that, for every shipment, the duration from the assigned start to the end point is at least equal to the sum of transportation and service times needed to cover all assigned nodes.

(iii) Loading constraints of each shipment: Constraints (9) and (10) ensure that shipment $v \in \mathbb{V}$ must visit the supplier $i \in \mathbb{N}'$ if any part type offered by the supplier is assigned to the shipment. Constraint (11) indicates that shipment $v \in \mathbb{V}$ must visit the warehouse if any of the suppliers is assigned to it. Constraints (12) and (13) indicate that, if part type k is assigned to shipment $v \in \mathbb{V}$, the inventory level of k will be replenished by the delivered packages after the finishing point f_{vt} of the shipment. Constraint (14) requires that the combined volume of all packages in each shipment does not exceed the truck's capacity.

(iv) Routing constraints for each shipment: Constraints (15) and (16) ensure that every shipment starts and ends at the warehouse, and that each supplier is visited no more than once per shipment. Constraint (17) is used to prevent sub-tours within each shipment.

(v) Constraints on the number of trucks: Constraints (18) to (20) make sure that, for any period $t \in \mathbb{T}$, $\theta_{vt} = 1$ if and only if shipment $v \in \mathbb{V}$ is occupied, which is equivalent to that the value of θ_{vt} has to be 1 for every t that is larger than the assigned starting point of shipment v and is smaller than or equal to the finishing point. Constraint (21) indicates that the total number of shipments occupied at any period should not be larger than the available number of trucks.

3.3. Feasibility conditions analysis

The above IP model requires setting the following four parameters to ensure that at least one feasible solution exists.

(1) Constant M

In constraints (10), (11) and (13), the constant M needs to be greater than the maximum possible value of $\sum_{k=1}^{K}\sum_{t=1}^{T}\beta_{vkt} \cdot r_{ki}$ and β_{vkt}. An upper bound of the two values can be calculated by $\lceil \frac{B}{\min_{k\in\mathbb{K}}\{b_k\}} \rceil$. Thus, the constant M should satisfy:

$$M > \lceil \frac{B}{\min_{k\in\mathbb{K}}\{b_k\}} \rceil \tag{23}$$

(2) Traveling time ε_{0i} from the warehouse to supplier $i \in \mathbb{N}'$

We assume that each shipment starts and finishes within the considered finite time horizon. For any supplier $i \in \mathbb{N}'$, a direct visit from the warehouse to the supplier takes a total traveling and operational time equal to $2 \cdot (\varepsilon_{0i} + o)$. Thus, the traveling time on

each edge ε_{0i}, $\forall i \in \mathbb{N}'$, should satisfy $\varepsilon_{0i} \le \frac{T-1}{2} - o$ such that at least a direct transportation between the warehouse and the supplier can be considered during current finite time horizon.

(3) Initial inventory p_{k1} of part type $k \in \mathbb{K}$

Consider the following two scenarios.

Case 1: $p_{k1} < l_k + \sum_{t=1}^{t_k^*} \delta_{kt}$, $\forall k \in \mathbb{K}$, $t_k^* = \lceil 2 \cdot \sum_{i=1}^{N} \varepsilon_{0i} + 2 \cdot o \rceil + 1$. The term t_k^* represents the number of periods needed to collect parts using direct point-to-point transportation. If the initial inventory level p_{k1} of part k is less than $l_k + \sum_{t=1}^{t_k^*} \delta_{kt}$, then even starting direct transportation at period 1 cannot replenish the stock in time to maintain the inventory above the lower bound l_k at any $t \in \mathbb{T}$. Hence, the IP model has no feasible solution in this scenario.

Case 2: $p_{k1} \ge l_k + \sum_{t=1}^{T} \delta_{kt}$, $\forall k \in \mathbb{K}$. Here, $\sum_{t=1}^{T} \delta_{kt}$ denotes the total demand for part k over the entire planning horizon. If the initial inventory can cover this total demand, constraint (4) is always satisfied, so part k can be excluded from the problem.

From these two cases, the proposed IP model is meaningful and feasible only if the starting inventory of each part $k \in \mathbb{K}$ satisfies the following inequality:

$$l_k + \sum_{t=1}^{t_k^*} \delta_{kt} \le p_{k1} \le l_k + \sum_{t=1}^{T} \delta_{kt} \tag{24}$$

Moreover, since the end of period T represents the beginning of the next planning horizon in practice, we modified constraint (4) in IP as the inequality (25) below to make it more implementable, where $\gamma_t = 1$ if $t = T$, otherwise $\gamma_t = 0$, and the lower bound l_k is the safety stock and is used to replace the real consumption $\sum_{t=1}^{t_k^*} \delta_{kt}$ in the future t_k^* periods, since the safety stock is usually defined by the largest possible consumption over a horizon in practice.

$$p_{kt} \ge l_k + \gamma_t \cdot (l_k + \delta_{kT}) \tag{25}$$

(4) Number of shipments $|\mathbb{V}|$

The size of the IP model, in terms of variables and constraints, is influenced by the number of available shipments $|\mathbb{V}|$, and the model may become computationally difficult to solve if $|\mathbb{V}|$ grows too large. We propose a greedy method (GM) to generate a proper value of $|\mathbb{V}|$, which is designed based on the rules of making decisions in practice. Algorithm 1 below is the pseudo-code of the GM algorithm.

Algorithm 1: Greedy method (GM) used in practice

Input: sets and parameters defined in IP
Output: a set of shipments $\mathbb{H}_{GM}$

initialization; $\mathbb{H}_{GM} \leftarrow \emptyset$
for $k = 1$ **to** K **do**
  $a_k \leftarrow \lceil \frac{2 \cdot l_k + \sum_{t=1}^{T} \delta_{kt} - p_{k1}}{h_k} \rceil$
  $\forall t \in \mathbb{T} \backslash \{1\}$, $r_{kt} \leftarrow p_{k1} - \sum_{z=1}^{t-1} \delta_{kz} - l_k$
  while $a_k \neq 0$ **do**
    generate a new shipment h
    $\exists \bar{t}$ holds $\forall (t \in \mathbb{T}) > \bar{t}$, $r_{kt} < 0$ and $r_{k\bar{t}} \ge 0$
    if $a_k - min(\lceil \frac{u_k - r_{k\bar{t}}}{h_k} \rceil, \lceil \frac{B}{b_k} \rceil) \ge 0$ **then**
      $w_{hk} \leftarrow min(\lceil \frac{u_k - r_{k\bar{t}}}{h_k} \rceil, \lceil \frac{B}{b_k} \rceil)$
    else
      $w_{hk} \leftarrow a_k$
    $\mathbb{H}_{GM} \leftarrow \mathbb{H}_{GM} \cup \{h\}$
    $r_{kt} \leftarrow r_{kt} + w_{hk} \cdot h_k$

For ease of expression, let $\mathbb{H}_{GM}$ represent the set of shipments generated by the GM algorithm, and $|\mathbb{H}_{GM}|$ will be the value for $|\mathbb{V}|$ used in the IP model. Denote h as each shipment generated by GM, and w_{hk} as the quantity of packages of part $k \in \mathbb{K}$ to be delivered in shipment h. The method first computes the minimum number of packages required for replenishment of each part type over the horizon, considering only the inventory constraint at the end, noted as a_k for all $k \in \mathbb{K}$. Based on the modified constraint (25), $a_k = \lceil \frac{2 \cdot l_k + \sum_{t=1}^{T} \delta_{kt} - p_{k1}}{h_k} \rceil$. Then, shipments are formed via point-to-point direct transportation for these packages. Define $r_{kt} = p_{k1} - \sum_{z=1}^{t-1} \delta_{kz} - l_k$ as the inventory of k at time period $t \in \mathbb{T} \backslash 1$ without replenishment.

4. Machine learning enhanced branch-and-price algorithm

This section presents a machine learning–enhanced branch-and-price algorithm (BPL) developed to efficiently solve the proposed IRP. Since the classic IRP can be reduced to our model, the proposed IRP variant is NP-hard. To improve tractability, Section 4.1 begins by reformulating the problem into a shipment-indexed integer programming master problem (IPMP) via Dantzig–Wolfe decomposition. Because enumerating all columns in IPMP, each of which refers to a feasible shipment, is impractical, a branch-and-price algorithm combining column generation and branch-and-bound is employed for exact solutions. To enhance practical

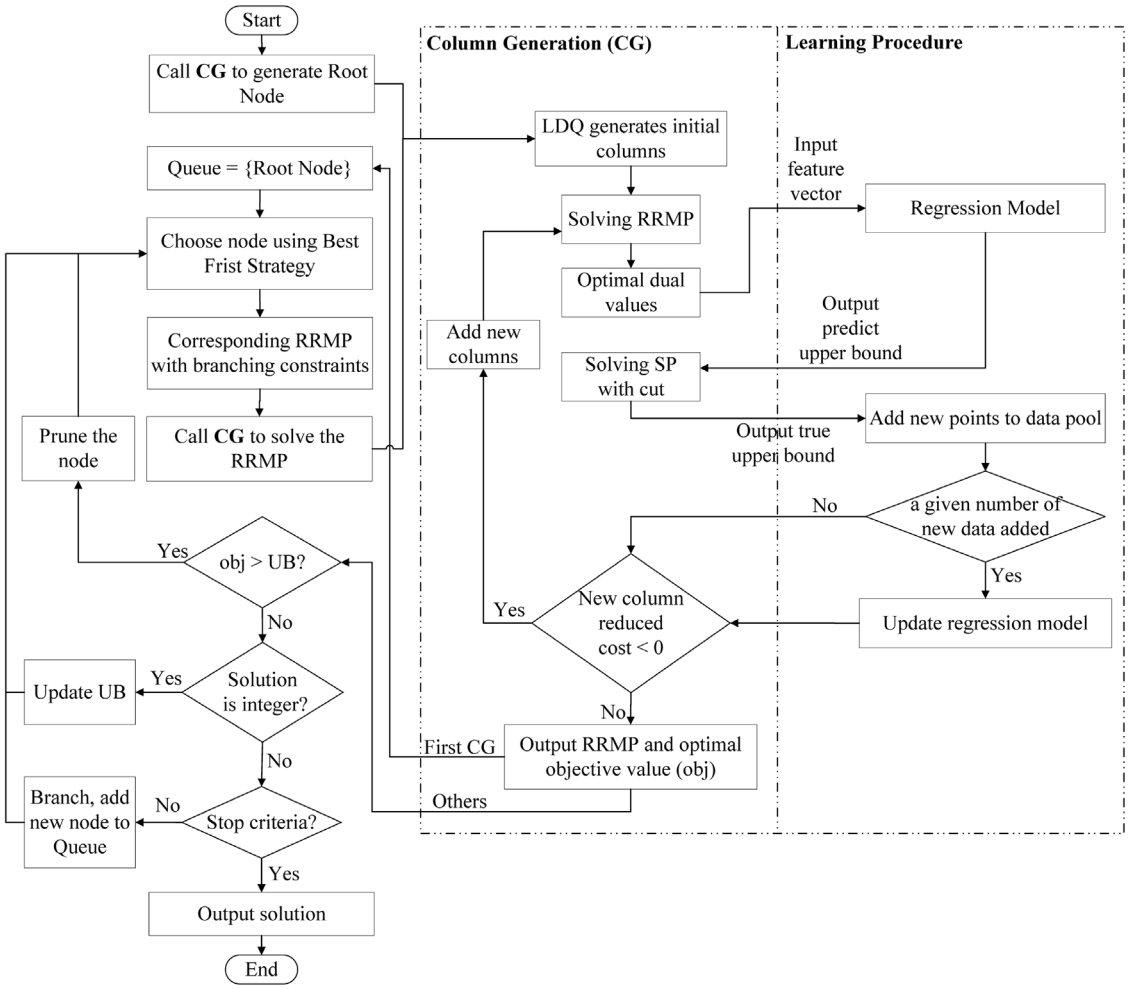


Fig. 3. The flow of BPL algorithm.

efficiency, an online regression method is integrated to predict upper bounds of subproblems during the column generation iterations, effectively speeding up the overall algorithm.

Fig. 3 illustrates the workflow of the proposed BPL algorithm. Initially, the linear relaxation of the IPMP is solved to optimality using column generation, establishing the root node for BPL. The column generation (CG) process is as follows: Section 4.2 introduces a heuristic based on the least delivery quantity (LDQ) rule to create the initial columns. Starting from the initial columns, a restricted master problem (RMP) containing a subset of IPMP's columns is defined. During each CG iteration, we solve the linear relaxation model of the RMP, denoted as RRMP, to optimality, and subproblems are formulated based on the associated optimal dual variables. As detailed in Section 4.3, new columns with negative reduced cost are generated by solving these subproblems, where an online regression model predicts each subproblem's upper bound and applies cuts for greater efficiency. The CG process terminates when no columns with negative reduced cost remain; the final solution is optimal for the RMP's relaxation and serves as the root node for the branch-and-price phase. Section 4.4 outlines the branch-and-price method, which explores branches from the root node using a hybrid branching strategy, repeatedly invoking a similar CG procedure at each node until a stopping condition is satisfied.

4.1. Problem reformulation

Let h represent a shipment in the proposed inventory routing problem. Using the parameters as in the IP model, denote $s_h, f_h \in \mathbb{T}$ as the scheduled start and finish time points of shipment h, and let w_{hk} be the quantity of packages of type $k \in \mathbb{K}$ assigned in this shipment. A shipment h is considered feasible if the total time required to visit the related suppliers is less than the planned time interval $f_h - s_h$, and the combined volume of all assigned packages does not exceed the capacity B of each truck. Define $\mathbb{H} = 1, 2, \ldots, H$ as the set of all such feasible shipments. The proposed IRP for packed auto parts with time-varying demands thus reduces to selecting a subset of feasible shipments from $\mathbb{H}$ that minimizes the combined inventory and transportation costs, while

ensuring that the inventory levels of each part at any time $t \in \mathbb{T}$ remain within the prescribed upper and lower bounds at the warehouse.

Introduce a binary variable x_h, where $x_h = 1$ if shipment $h \in \mathbb{H}$ is chosen, and $x_h = 0$ otherwise. For all $h \in \mathbb{H}$, $k \in \mathbb{K}$, and $t \in \mathbb{T}$, define w_{hkt} as the number of packages of type k delivered by shipment h at the start of period t. Specifically, for any $t \geq f_h$, $w_{hkt} = w_{hk}$; otherwise, $w_{hkt} = 0$. For each feasible shipment $h \in \mathbb{H}$, let $\mathbb{N}_h$ represent the set of suppliers providing the parts included in h, and $TR(\mathbb{N}_h)$ be the total travel and handling time for that shipment. Incorporating constraints (2) and (3) from the IP model into the part on the inventory cost in the objective function, the total costs for the selected shipments can be expressed as:

$$C_1 \cdot \left[\sum_{t=1}^{T} \sum_{k=1}^{K} (p_{k1} - \frac{1}{2} \sum_{z=1}^{t-1} \delta_{kz} - \frac{1}{2} \sum_{z=1}^{t} \delta_{kz}) + \sum_{h=1}^{H} \sum_{k=1}^{K} (T - f_h + 1) \cdot w_{hk} \cdot h_k \cdot x_h \right] + C_2 \cdot \sum_{h=1}^{H} TR(\mathbb{N}_h) \cdot x_h \tag{26}$$

As shown in Eq. (26), the total inventory cost contains the inventory cost deriving from the initial inventory and the parts delivered by different shipments. Define the known parts in the total inventory cost as

$$\Omega = C_1 \cdot \sum_{t=1}^{T} \sum_{k=1}^{K} (p_{k1} - \frac{1}{2} \sum_{z=1}^{t-1} \delta_{kz} - \frac{1}{2} \sum_{z=1}^{t} \delta_{kz}) \tag{27}$$

Then the inventory and routing cost related to each of the selected shipments h is denoted as:

$$\varphi_h = C_1 \sum_{z=f_h}^{T} \sum_{k=1}^{K} w_{hk} \cdot h_k + C_2 \cdot TR(\mathbb{N}_h) \tag{28}$$

Using the sets and cost function, the proposed problem can be reformulated based on Dantzig–Wolfe decomposition as the following integer programming model IPMP:

$$\text{(IPMP)} \quad \min \sum_{h \in \mathbb{H}} \varphi_h \cdot x_h + \Omega \tag{29}$$

$$\text{s.t. } p_{k1} - \sum_{z=1}^{t-1} \delta_{kz} + \sum_{h \in \mathbb{H}} w_{hkt} \cdot h_k \cdot x_h \geq l_k + \gamma_t \cdot (l_k + \delta_{kT}), \forall k \in \mathbb{K}, t \in \mathbb{T} \backslash \{1\} \tag{30}$$

$$p_{k1} - \sum_{z=1}^{t-1} \delta_{kz} + \sum_{h \in \mathbb{H}} w_{hkt} \cdot h_k \cdot x_h \leq u_k, \forall k \in \mathbb{K}, t \in \mathbb{T} \backslash \{1\} \tag{31}$$

$$\sum_{h \in \mathbb{H}} \theta_{ht} \cdot x_h \leq G, \forall kt \in \mathbb{T} \tag{32}$$

$$x_h \in \{0, 1\} \tag{33}$$

4.2. Initial columns generation

The reformulated IPMP is addressed using a branch-and-price framework integrated with column generation. Initially, the linear relaxation of the IPMP is solved optimally through the CG procedure, where a restricted master problem (RMP) is iteratively refined: its linear relaxation is solved, and additional columns are introduced by solving the associated pricing subproblems. To initiate this process, this section designs a heuristic method based on the Least Delivery Quantity (LDQ) rule, which generates initial feasible columns for the RMP.

The LDQ algorithm considers milk-run routes based on the GM algorithm. As previously outlined, a minimum of a_k packages of each type $k \in \mathbb{K}$ must be dispatched within the planning horizon $\mathbb{T}$, where $a_k = \lceil \frac{2 \cdot l_k + \sum_{t=1}^{T} \delta_{kt} - p_{k1}}{h_k} \rceil$ ensures satisfaction of inventory constraints stated in (25). Let r_{kt} denote the inventory availability of part k at the start of period $t \in \mathbb{T}$. Define $\mathbb{H}_{LDQ}$ as the set of all feasible shipments constructed by the LDQ approach, each representing a candidate column in the IPMP model. The procedure and pseudo-code for generating these initial columns using the LDQ heuristic are provided below:

Step 1. Initialize $\mathbb{H}_{LDQ} \leftarrow \emptyset$; for all $k \in \mathbb{K}, t \in \mathbb{T} \backslash \{1\}$, let $r_{kt} \leftarrow p_{k1} - \sum_{z=1}^{t-1} \delta_{kz} - l_k$. There are $A = \sum_{k \in \mathbb{K}} a_k$ packages of all types to be delivered over the horizon. Let $\mathbb{A} = \{1, 2, \ldots, A\}$ be the set of packages to be delivered.

Step 2. For each part type $k \in \mathbb{K}$, calculate the current deadline of parts arrival, denoted as t_k, which is defined as the time when the current available inventory is consumed out.

Step 3. Arrange the packages in $\mathbb{A}$ in increasing order based on t_{k_a}, where $k_a \in \mathbb{K}$ is the part type of package $a \in \mathbb{A}$, and t_{k_a} represents the current deadline of delivering a calculated in Step 2. Let $\mathbb{M} = \{1, 2, \ldots, |A|\}$ represent the sorted set.

Step 4. Generate a new shipment h, initialize the assigned packages as $\mathbb{A}_h = \{m_1\}$ and the related nodes to visited as $\mathbb{N}_h = \{0, I_{m_1}, 0\}$. Let f_h and e_h be the finishing time point and the total transportation time of the shipment, respectively. Let $f_h \leftarrow t_{k_{m_1}}$, $e_h \leftarrow e(\mathbb{N}_h) = \sum_{i=1}^{\|\mathbb{N}_h\|-1} \epsilon_{i,i+1}$, $d_h \leftarrow \lceil e_h \rceil$, $w_{hkm_1} \leftarrow 1$, and $w_{hk} = 0$, $\forall k \neq k_{m_1}$.

Step 5. Insert as many of the remaining packages to $\mathbb{A}_h$ as possible subject to the truck capacity constraint and upper bound constraint on inventory level.

Step 6. Update $\mathbb{H}_{LDQ}$, $\mathbb{A}$ and p_{kt} as follows: $\mathbb{H}_{LDQ} \leftarrow \mathbb{H}_{LDQ} \cup h$, $\mathbb{A} \leftarrow \mathbb{A} \backslash \mathbb{A}_h$, $\forall k \in \mathbb{K}, \forall (t \in \mathbb{T} \backslash \{1\}) \geq f_h, r_{kt} \leftarrow r_{kt} + w_{hk} \cdot h_k$. If $\mathbb{A} \neq \emptyset$, then repeat *Step 2* to *Step 6*, stop otherwise.

Algorithm 2: LDQ to generate the initial solution

$\mathbb{H}_{LDQ} \leftarrow \emptyset$; $\mathbb{A} = \{1, 2, \dots, A\}, A \leftarrow \sum_{k\in\mathbb{K}} a_k$; $\forall k \in \mathbb{K}, t \in \mathbb{T}\backslash\{1\}, r_{kt} \leftarrow p_{k1} - \sum_{z=1}^{t-1} \delta_{kz} - l_k$;
while $\mathbb{A} \neq \emptyset$ **do**
  for $k \in \mathbb{K}$ **do**
    if $\delta_{kT} \le r_{kT} < l_k + \delta_{kT}$, **then**
      $t_k \leftarrow T$
    else if $\exists\ \bar{t}$ *holds* $\forall (t \in \mathbb{T}\backslash\{1\}) > \bar{t}, r_{kt} < 0 \wedge r_{k\bar{t}} \ge 0$, **then**
      $t_k \leftarrow \bar{t}$
  Sort packages in $\mathbb{A}$ by ascending order of t_{k_a}, denote the sorted set as $\mathbb{M} = \{m_1, m_2, \dots, m_{|\mathbb{A}|}\}$
  Initialize $\mathbb{A}_h \leftarrow \{m_1\}$, $\mathbb{N}_h \leftarrow \{0, I_{m_1}, 0\}$, finishing time $f_h \leftarrow t_{k_{m_1}}$, transportation time $e_h \leftarrow e(\mathbb{N}_h) = \sum_{i=1}^{\|\mathbb{N}_h\|-1} \varepsilon_{i,i+1}$, $d_h \leftarrow \lceil e_h \rceil$, $w_{hk_{m_1}} \leftarrow 1$, and $w_{hk} = 0, \forall k \neq k_{m_1}$
  for $z = 2$ **to** $\|\mathbb{M}\|$ **do**
    $k^* \leftarrow k_{m_z}$
    if $\sum_{m\in\mathbb{A}_h} b_{k_m} + b_{k^*} \le B \wedge r_{k^* f_h} + (w_{hk^*} + 1) \cdot h_{k^*} \le u_{k^*}$ **then**
      $\mathbb{N}_h \leftarrow \mathbb{N}_h[:-1] \cup \{I_{m_z}\} \cup \{0\}$
      if $t_{m_z} = f_h, d_h < f_h \vee I_{m_z} \in \mathbb{N}_h$ **then**
        $\mathbb{A}_h \leftarrow \mathbb{A}_h \cup \{m_z\}$, $w_{hk^*} \leftarrow w_{hk^*} + 1$
        if $I_{m_z} \notin \mathbb{N}_h$ **then**
          $\mathbb{N}_h \leftarrow \mathbb{N}_h[:-1] \cup \{I_{m_z}\} \cup \{0\}$
  for $z = 2$ **to** $\|\mathbb{M}\|$ **do**
    $k^* \leftarrow k_{m_z}$
    if $\sum_{m\in\mathbb{A}_h} b_{k_m} + b_{k^*} \le B \wedge r_{k^* f_h} + (w_{hk^*} + 1) \cdot h_{k^*} \le u_{k^*}$ **then**
      $\mathbb{N}_h \leftarrow \mathbb{N}_h[:-1] \cup \{I_{m_z}\} \cup \{0\}$
      if $m_z \notin \mathbb{A}_h \wedge d_h < f_h$ **then**
        $\mathbb{A}_h \leftarrow \mathbb{A}_h \cup \{m_z\}$, $w_{hk^*} \leftarrow w_{hk^*} + 1$, $\mathbb{N}_h \leftarrow \mathbb{N}_h[:-1] \cup \{I_{m_z}\} \cup \{0\}$
  $\mathbb{H}_{LDQ} \leftarrow \mathbb{H}_{LDQ} \cup \{h\}$, $\mathbb{A} \leftarrow \mathbb{A}\backslash\mathbb{A}_h$
  $\forall k \in \mathbb{K}, \forall (t \in \mathbb{T}\backslash\{1\}) \ge f_h, r_{kt} \leftarrow r_{kt} + w_{hk} \cdot h_k$

4.3. Solving the subproblems with online machine learning

During each iteration of the CG process, the optimal solution to the linear relaxation of the RMP is computed, along with the values of its associated dual variables. For every combination of $k \in \mathbb{K}$ and $t \in \mathbb{T}\backslash 1$, denote the dual variables for constraints (30) and (31) in the RRMP as π^1_{kt} and π^2_{kt}, respectively. Let ψ_t represent the dual variable corresponding to constraint (32), for all $t \in \mathbb{T}$. Then, for any column that has not been added to the incumbent RRMP, the reduced cost is

$$\hat{\varphi}_h = \varphi_h - \sum_{t=2}^{T}\sum_{k=1}^{K} w_{hk} \cdot h_k \cdot (\pi^1_{kt} - \pi^2_{kt}) + \sum_{t=1}^{T} \theta_{ht} \cdot \psi_t \tag{34}$$

Based on Eq. (34), the subproblem is formulated to generate new feasible shipments such that the reduced cost is minimized. In addition to parameters already introduced in the IP model, define the following decision variables: a non-negative integer w_k indicating the number of packages of type $k \in \mathbb{K}$ assigned to the shipment; integers s and f representing the planned start and end periods of the shipment within the finite planning horizon. Let θ_t be a binary variable indicating whether the shipment is active at the start of period $t \in \mathbb{T}$, where $\theta_t = 1$ denotes it is active, and $\theta_t = 0$ otherwise. Similarly, binary variable $x_{ij} = 1$ if the route includes edge $(i, j) \in \mathbb{E}$, and $x_{ij} = 0$ if not. Let $y_i = 1$ represent that node $i \in \mathbb{N}$ is visited by the shipment, and $y_i = 0$ otherwise. Additionally, let σ_i be a non-negative variable that captures the visiting order of supplier $i \in \mathbb{N}'$. The subproblem is thus formulated as the following SP model:

$$(SP) \quad \min \sum_{z=f}^{T}\sum_{k=1}^{K} w_k \cdot h_k \cdot (C_1 - \pi^1_{kt} + \pi^2_{kt}) + C_2 \sum_{(i,j)\in\mathbb{E}} \varepsilon_{ij} \cdot x_{ij} + \sum_{t=1}^{T} \theta_t \cdot \psi_t \tag{35}$$

$$\text{s.t. } y_i \le \sum_{k=1}^{K} w_k \cdot r_{ki}, \forall i \in \mathbb{N}' \tag{36}$$

$$M \cdot y_i \ge \sum_{k=1}^{K} w_{k^* ki}, \forall i \in \mathbb{N}' \tag{37}$$

$$M \cdot y_0 \geq \sum_{i=1}^{N} y_i \tag{38}$$

$$\sum_{j=0, j\neq i}^{N} x_{ij} = y_i, \forall i \in \mathbb{N} \tag{39}$$

$$\sum_{j=0, j\neq i}^{N} x_{ij} - \sum_{j=0, j\neq i}^{N} x_{ji} = 0, \forall i \in \mathbb{N} \tag{40}$$

$$\sigma_i - \sigma_j + N \cdot x_{ij} \leq N - 1, \forall i, j \in \mathbb{N}', i \neq j \tag{41}$$

$$f - s \geq \sum_{(i,j)\in\mathbb{E}} \epsilon_{ij} \cdot x_{ij} + \sum_{i=0}^{N} y_i \cdot o \tag{42}$$

$$\sum_{k=1}^{K} w_k \cdot b_k \leq B \tag{43}$$

$$\theta_t \cdot t \leq f, \forall t \in \mathbb{T} \tag{44}$$

$$(1 - \theta_t) \cdot M + \theta_t \geq s + 1, \forall t \in \mathbb{T} \tag{45}$$

$$\sum_{t=1}^{T} \theta_t \geq f - s \tag{46}$$

$$\theta_t, y_i, x_{ij} \in \{0, 1\}, w_k, s, f, \sigma_i \in Z^+ \tag{47}$$

The SP model becomes an integer programming model when the value of f is given in the objective function. Thus, we transform the SP model into $T - \min_{k\in\mathbb{K}}\{t_k^*\}$ decomposed subproblems by enumerating the possible value of variable f. Even though the integer programming model of each decomposed subproblem becomes relatively easier to solve based on a given value of f, the efficiency of solving the subproblems is still the key for the implementation of the branch-and-price algorithm in practice. This is because subproblems must be repeatedly formulated and solved to optimality throughout the CG process, which itself is invoked multiple times during the entire BPL procedure.

To reduce the entire computational time, an online regression method is designed to predict the upper bound of the subproblems. Fig. 4 illustrates the learning procedure of the online regression. During column generation in the BnP procedure, a feature vector is generated and defined by the parameters related to each of the formulated subproblems. If the incumbent data pool $\mathbb{D}$ contains fewer than the specified number D of data points, the subproblem is directly solved to obtain its optimal objective value $\hat{\phi}$. Otherwise, a regression model is trained using D data samples from $\mathbb{D}$ to estimate an upper bound of the subproblem, denoted as $\hat{F}(\phi)$. Based on the predicted upper bound, a cutting constraint $\hat{\phi} \leq \hat{F}(\phi)$ will be added to the subproblem to solve it more efficiently, since it could eliminate the suboptimal or infeasible solutions of the searching space. In either case, the subproblem's feature vector, along with its optimal objective value, which represents the actual upper bound, constitutes a new data sample that is added to the data pool $\mathbb{D}$. The data pool will be updated by adding the new data, and the oldest data point will be deleted when more than D points are stored. The optimal weight parameters in the regression model, denoted as w^T, are obtained by minimizing the loss function $L(\mathbb{D})$ and are updated every episode, which is defined as a period of every $\mathcal{Q}$ new data points added to the pool. The process goes back to BnP after solving the subproblem and updating the data pool and regression model.

The features, data and loss function of the online regression process are described in detail below, along with how the predicted upper bound contributes to the column generation procedure.

4.3.1. Features selection

Since there are many dual variables in the problem, it is extremely hard to achieve a reasonable prediction directly using such a high-dimensional vector. We transform the dual variables π_{kt}^1, π_{kt}^2 and ψ_t, for all $k \in \mathbb{K}$ and $t \in \mathbb{T}$, into scalars ϕ_1, ϕ_2, ϕ_3 as follows:

$$\phi_1 = \frac{\sum_{t=f}^{T}\sum_{k=1}^{K} \pi_{kt}^1}{T - f + 1} \tag{48}$$

$$\phi_2 = \frac{\sum_{t=f}^{T}\sum_{k=1}^{K} \pi_{kt}^2}{T - f + 1} \tag{49}$$

$$\phi_3 = \frac{\sum_{t=f}^{T} \psi_t}{T - f + 1} \tag{50}$$

Define $\phi = [\phi_1, \phi_2, \phi_3, f]$ to be the feature vector for the online regression model, where f is the finishing point of the shipment in a decomposed subproblem as defined in SP. More feature vectors are discussed and compared in Section 5.4.

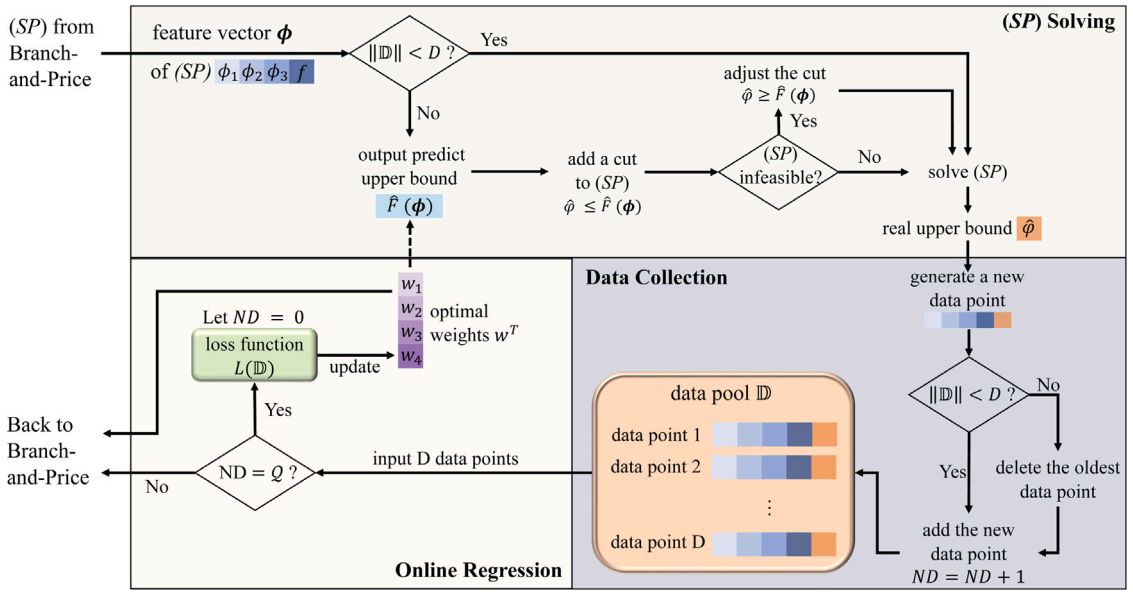


Fig. 4. The framework of learning procedure in the BPL algorithm.

4.3.2. Data collection

Starting from the first iteration during CG process, the dual variable values of RRMP are collected and transformed into the features. The feature vectors and the corresponding optimal objective values of the decomposed SPs consist of the data set for training the online regression model. Let $\mathbb{D}$ be the set of data points collected, and each data point $d \in \mathbb{D}$ consists of the feature vector ϕ^d and the corresponding optimal objective value of the subproblem, denoted as $\hat{\varphi}^d$. Based on the collected data, a regression model trained can estimate the upper bound of the subproblem, which serves as a cut to accelerate its solution process. After solving each decomposed subproblem, a new data point is generated and added to the dataset $\mathbb{D}$ if the size of $\mathbb{D}$ is less than D, where D is a constant representing the minimum number of samples needed to train the regression model. To avoid an explosion on the scale of the dataset, we keep $|\mathbb{D}|$ as a fixed constant by deleting the earliest data point in the set when each new data is added after $|\mathbb{D}| \geq D$.

4.3.3. Loss function

Define $\hat{F}(\phi) = w^T \cdot \phi$ as the function to predict the upper bound of the subproblems' optimal objective values, where $\hat{F}$ is the prediction value and w is the prediction vector. The predicted upper bound $\hat{F}$ is required to be larger than the real objective value $\hat{\varphi}$, with closer approximations being preferable. Thus, let $\eta = \hat{F}(\phi^d) - \hat{\varphi}^d$, and the loss function $l(\hat{F}(\phi), \hat{\varphi})$ is given as follows:

$$l(\hat{F}(\phi^d), \hat{\varphi}^d) = \begin{cases} \kappa_1 \cdot \eta & , \eta < 0 \\ 0 & , \eta \in [0, \xi] \\ \kappa_2 \cdot \eta & , \eta > \xi \end{cases} \tag{51}$$

where κ_1, κ_2 are the loss penalty parameters, and ξ is a pre-determined tolerable gap between the prediction value and the real one.

Considering that the data from the most recent iterations should be more influential to the prediction, a discount factor ρ is introduced to the losses (Václavík et al., 2018). For every data point $d \in \mathbb{D}$ generated from the ith iteration of the CG procedure, the discount factor ρ_d is defined as

$$\rho_d = \rho(\lambda, a, b, m)_i = \lambda^{a + \frac{(i-1)(b-a)}{m-1}}, \tag{52}$$

where λ, a, b are constants and m represents the index of the current iteration.

Let e_d^-, e_d^+ represent the negative and positive deviation of the prediction value from the real value, respectively. Let te_d^+ be the gap between e_d^+ and a tolerance parameter ξ. Taking the discount factor into consideration, the regression model is formulated as the linear programming model on minimizing the empirical loss function $L(\mathbb{D})$v as follows.

$$\min \quad L(\mathbb{D}) = \sum_{d \in \mathbb{D}} \rho_d \cdot (\kappa_1 \cdot e_d^- + \kappa_2 \cdot te_d^+) \tag{53}$$

$$\text{s.t.} \quad w^T \cdot \phi^d - e_d^+ + e_d^- = \hat{\varphi}^d, \forall d \in \mathbb{D} \tag{54}$$

$$te_d^+ \geq e_d^+ - \xi, \forall d \in \mathbb{D} \tag{55}$$

$$e_d^-, e_d^+, te_d^+ \geq 0, \forall d \in \mathbb{D} \tag{56}$$

After each iteration of CG procedure, the dataset $\mathbb{D}$ and its associated regression model are updated to improve predictions of the subproblems' objective upper bounds. The predicted upper bound $\hat{F}(\phi_h)$ is added as a cutting constraint, namely $\hat{\varphi}_h \leq \hat{F}(\phi_h)$, into the model while solving the subproblem. Additionally, the subproblem with the above upper bounding constraint may become infeasible if the predicted value is occasionally lower than the subproblem's optimal objective value. In such cases, we modify the constraint to $\hat{\varphi}_h \geq \hat{F}(\phi_h)$ and re-solve the subproblem to generate new columns. This lower bounding constraint has also proven to be effective in our experiments.

4.4. Branch and price framework

By applying CG to solve the linear relaxation of the IPMP, the optimal solution obtained from the final iteration of the RRMP is also optimal for the relaxed IPMP. However, this solution is typically fractional and does not satisfy the integer constraints of the original problem. To find an optimal integer solution, a branch-and-price framework is initiated using this relaxed solution as the root node. Section 4.4.1 describes a hybrid branching strategy, and Section 4.4.2 outlines the search and bounding strategy used within the BnP algorithm. Since the CG procedure with online regression would be called repeatedly, Section 4.4.3 introduces knowledge-sharing rules along the branching tree for the regression model. To make the proposed algorithm more tractable in practice, Section 4.4.4 summarizes the acceleration strategies used in the proposed algorithm and the early termination criteria.

4.4.1. Hybrid branching strategy

In order to efficiently identify the optimal integer solution, we design a hybrid branching strategy based on the aggregate value of all variables at the current node. Starting from the root node, define x_h^*, $\forall h \in \mathbb{H}$, as the optimal solution of the IPMP linear relaxation corresponding to each node on the branching tree. New branches are generated according to the following rules:

Branching Rule 1: If the value of $\sum_{h\in\mathbb{H}} x_h^*$ is an integer, then divide the problem of the current node into three sub-nodes by introducing branching constraint $\sum_{h\in\mathbb{H}} x_h \leq \sum_{h\in\mathbb{H}} x_h^* - 1$, $\sum_{h\in\mathbb{H}} x_h = \sum_{h\in\mathbb{H}} x_h^*$ and $\sum_{h\in\mathbb{H}} x_h \geq \sum_{h\in\mathbb{H}} x_h^* + 1$, respectively. Otherwise, divide the problem into two sub-nodes by introducing $\sum_{h\in\mathbb{H}} x_h \leq \lfloor \sum_{h\in\mathbb{H}} x_h^* \rfloor$ and $\sum_{h\in\mathbb{H}} x_h \geq \lceil \sum_{h\in\mathbb{H}} x_h^* \rceil$, respectively.

Branching Rule 2: If the current node already contains a branching constraint from Rule 1 and satisfies $\sum_{h\in\mathbb{H}} x_h^*$ is an integer, then choose the two most infeasible variables, denoted by x_1^* and x_2^*, whose values are two closest to 0.5. If $0 < x_1^* + x_2^* < 1$, divide the problem of the current node into two sub-nodes by introducing new constraint $x_1^* + x_2^* = 0$ and $x_1^* + x_2^* \geq 1$, respectively. If $x_1^* + x_2^* = 1$, then divide the problem by $x_1^* = 1$ and $x_2^* = 1$, respectively. If $1 < x_1^* + x_2^* < 2$, then divide the problem by $x_1^* + x_2^* \leq 1$ and $x_1^* + x_2^* = 2$, respectively.

4.4.2. Searching and bounding strategy

In our algorithm, the Best First strategy is applied to choose the next node for exploration. Namely, in each iteration, we pre-solve the RRMP model with the branching constraint corresponding to each of the generated sub-nodes and choose the one with the smallest objective value to call the CG algorithm.

The RRMP model corresponding to the root node can be transformed into an integer programming model by letting every variable x_h as an integer, and its optimal objective value serves as an upper bound for the original problem, denoted as UB. Since this root node model includes only a subset of all possible columns, it can be solved efficiently using a commercial solver.

The search begins at the root node and proceeds along the branches generated using the branching strategy outlined in Section 4.4.1. For each node n explored, the corresponding RRMP model with the branching constraint is denoted as $RRMP(n)$. The column generation algorithm with online regression is applied to solve $RRMP(n)$, and its optimal objective value, $Obj(n)$, serves as a lower bound $LB(n)$ for all descendant nodes of n. If $LB(n) \geq UB$ or $RRMP(n)$ is infeasible, then the subtree rooted at n can be safely discarded. If $RRMP(n)$ yields an optimal integer solution and $Obj(n) < UB$, then update the value of UB as $UB = Obj(n)$.

4.4.3. Sharing knowledge along the branching tree

When solving each problem corresponding to each child node on the branching tree by CG, it is not efficient to start a new data set for online regression, and some child nodes cannot effectively update the regression model for prediction due to insufficient data points. Considering the fact that there is only one more constraint in each child node compared to its parent node, we design the following three rules for sharing data among the nodes along the branching tree as follows:

(i) A child node can only use data from its parent nodes.

(ii) Data shared from parent nodes must be filtered by branching constraints.

(iii) The data in the regression model are gradually removed through the branching process such that the data set remains a given scale.

4.4.4. Acceleration strategy and stop criteria

Usually, the branch-and-price procedure stops when no new branches can be generated and explored, and the optimal integer solution can be found for the origin problem. The entire procedure could take a very long time, and a tailing-off problem is commonly concerned in column generation-based algorithms where the optimal objective function value improves only slightly in the later iterations. To make the algorithm more efficient in practical use, we summarize the three types of acceleration strategies applied in the proposed BPL algorithm as follows. Note that the early termination rules make the BPL algorithm more efficient in practice but stop at a near-optimal solution instead of solving the problem exactly.

(i) Hybrid branching strategy. As described in Section 4.4.1, two branching rules are designed for generating new nodes in our algorithm. Differing from the traditional strategy that branches on a single variable, the proposed hybrid strategy sharply reduces the size of the branching tree and enhances searching efficiency.

(ii) Subproblem upper bound prediction by online regression. The subproblems SP are formulated and solved to optimality repeatedly in the entire BPL procedure. To accelerate the calculation of the subproblems, an online regression model trained on dual variable values is employed to predict an upper bound prior to solving each subproblem, as outlined in Section 4.3. Instead of solving the subproblems directly, a cut is added to the subproblems using the predicted upper bound, and the computational efficiency of solving the subproblems is improved.

(iii) Early termination (ET) rules. In addition to the common stop criteria, two early termination rules are used in our algorithm to avoid the tailing-off of column generation.

ET Rule 1: If the objective value of the best integer solution remains unchanged after exploring a predefined number of nodes, the algorithm is terminated, and the current best solution is returned.

ET Rule 2: When exploring each node on the branching tree, let UB be the best upper bound generated, and GLB be the current global lower bound. The entire algorithm terminates when the relative difference between the upper and lower bounds, defined as $\frac{UB-GLB}{GLB}$, is smaller than a given tolerance parameter ϵ.

Let $\mathbb{N}_{BnP}$ be the set of remaining nodes to be explored in a given iteration of the BPL algorithm. As described in Section 4.4.2, the upper bound UB is the objective value of the best integer solution currently generated on the branching tree. $LB(n)$ is the lower bound of each node $n \in \mathbb{N}_{BnP}$, and the global lower bound GLB equal to the smallest $LB(n)$ among all nodes in $\mathbb{N}_{BnP}$. For each node $n \in \mathbb{N}_{BnP}$, $LB(n)$ is obtained by solving the branching subproblem corresponding to its parent node using the column generation algorithm with online regression, and, during each iteration of the CG process as well as the last iteration with the optimal solution, the lower bound could be calculated as follows:

$$LB(n) = \Omega + \sum_{h \in \mathbb{H}_n} \hat{\varphi}_h + \sum_{t=1}^{T} \sum_{k=1}^{K} (\pi_{kt}^1 \cdot \omega_{kt}^1 - \pi_{kt}^2 \cdot \omega_{kt}^2) + \sum_{t=1}^{T} \psi_t \cdot G \tag{57}$$

where $\forall k \in \mathbb{K}$, $\forall t \in \mathbb{T} \backslash \{1\}$, π_{kt}^1, π_{kt}^2 and ψ_t are the dual variable values obtained by solving the restricted master problem of CG, $\mathbb{H}_n$ represents the set of columns selected in the iteration of CG, and let $\omega_{kt}^1 = l_k + \gamma_t \cdot (l_k + \delta_{kT}) - p_{k1} + \sum_{z=1}^{t-1} \delta_{kz}$, $\omega_{kt}^2 = u_k - p_{k1} + \sum_{z=1}^{t-1} \delta_{kz}$.

5. Numerical experiments

To evaluate the effectiveness of the proposed BPL algorithm, this section presents numerical experiments results using randomly generated test instances. The proposed IP model, RRMP and SP models are solved by CPLEX 20.1.0, respectively. The solver and the proposed algorithms are carried out in Python on a machine equipped with a 1.80 GHz processor and 16 GB of RAM.

5.1. Instances generation

For the proposed IRP problem, we generate instances of varying sizes to test the performance of the BPL algorithm. The scale of the instances is determined by the configuration of the number of part types, K, the number of suppliers, N, and the number of periods on the finite planning horizon, T. We first define the values and combinations of these parameters as follows:

- Number of part types: $K = 5, 10, 15, 20$.
- Number of suppliers (N), which depends on the chosen value of K: for $K = 5$, let $N = 2$; for $K = 10$, let $N = 5, 10$; for $K = 15$, let $N = 3, 10$; for $K = 4$, let $N = 10, 20$.
- The finite time horizon: $T = 8, 24, 48$ refers to 8, 24 and 48 h.

It is worth noting that the values chosen for K, N, and T are based on insights gained from a real-world case study conducted at an assembly plant located in Shanghai. For a given configuration of (K, N, T), 10 instances are generated as a group, in which some of the parameters are decided according to practical values, and the other parameters are generated randomly from a given interval, with random probability consistent with a uniform distribution. Thus, according to the values of K, N, T, 21 groups of instances are generated in total, and the details of the parameters are described below.

Geographical locations of suppliers and warehouses are randomly generated in a circular area with a radius of 120 km. The Euclidean distance is taken as the distance between any two vertices. The truck operates at an average speed of 60 km/h, with an average service time of 0.2 h at each supplier, and its capacity B is 49.77 m^3. The inventory cost per unit time C_1 is 0.125/unit h. The shipping cost per unit time C_2 is 93.6/h. The pre-determined TAKT is 2 min, and one period contains 30 TAKT. Part consumption per period for each type, denoted as δ_{kt}, is generated by first drawing a random integer $u_{kl} \in [0, 4]$, representing the quantity from

Table 3
Experimental results on comparing the proposed BPL algorithm to the global lower bound (GLB), solving IP by CPLEX (IP), the practical greedy method (GM) and a Genetic Algorithm (GA).

Group	K	N	T	Average relative gaps				Time (s)		
				BPL-GLB	BPL-IP	BPL-GM	BPL-GA	IP	BPL	GA
1	5	2	8	7.73%	3.37%	−28.60%	−72.78%	10.04(0)	22.63	14.52
2	5	2	24	3.88%	−10.35%	−22.54%	−55.31%	3600(7)	82.52	36.90
3	5	2	48	5.03%	−7.45%	−25.44%	−46.99%	3600(10)	128.38	67.14
4	10	5	8	6.35%	−3.76%	−25.42%	−74.63%	3600(10)	22.89	27.81
5	10	5	24	3.46%	−18.20%	−22.17%	−61.99%	3600(10)	121.80	69.18
6	10	5	48	3.56%	−9.76%	−23.88%	−53.69%	3600(10)	232.68	124.49
7	10	10	8	7.11%	−7.44%	−21.38%	−78.90%	3600(10)	49.42	30.89
8	10	10	24	7.15%	−3.00%	−20.76%	−66.54%	3600(10)	123.42	75.79
9	10	10	48	2.28%	−28.33%	−24.75%	−52.75%	3600(10)	248.94	135.65
10	15	3	8	4.19%	−4.96%	−24.78%	−73.73%	3600(10)	114.00	39.26
11	15	3	24	6.40%	−19.31%	−25.00%	−56.83%	3600(10)	187.17	98.44
12	15	3	48	4.52%	−39.63%	−40.21%	−58.58%	3600(10)	410.25	176.77
13	15	10	8	5.41%	−11.31%	−24.95%	−81.48%	3600(10)	107.52	42.57
14	15	10	24	2.92%	−35.84%	−23.39%	−58.63%	3600(10)	131.00	106.39
15	15	10	48	6.53%	−36.79%	−37.93%	−50.63%	3600(10)	561.58	195.46
16	20	10	8	3.12%	−0.87%	−16.03%	−77.13%	3600(10)	133.25	53.52
17	20	10	24	1.04%	−28.98%	−31.22%	−55.86%	3600(10)	211.80	133.88
18	20	10	48	4.89%	−45.77%	−30.39%	−46.99%	3600(10)	528.60	241.13
19	20	20	8	2.91%	−17.65%	−42.38%	−85.37%	3600(10)	196.71	58.03
20	20	20	24	3.50%	−36.80%	−50.64%	−59.19%	3600(10)	523.72	144.56
21	20	20	48	0.82%	−47.14%	−52.77%	−44.34%	7200(10)	717.06	257.01
Average				4.42%	−19.52%	−29.87%	−62.49%	3771.91	231.21	101.40

the BOMs used at TAKT l for various customer orders, and then computing $\delta_{kt} = \sum_{l=1}^{30} u_{kl}$. The quantity per package h_k for each part type is a random integer from the range $[10, 100]$, while the volume per package b_k is randomly selected from the interval $[0.01B, 0.1B]$. The lower bound of each part type $l_k = \lceil 2 \cdot (\sum_{i=1}^{N} \epsilon_{0i} \cdot r_{ki} + o) \rceil \cdot 30 \cdot \epsilon_k$ and ϵ_k represent the maximum possible quantity of part k used at a TAKT. The initial inventory of each part type p_{k1} is an integer generated from $[l_k + \sum_{t=1}^{t_k^*} \delta_{kt}, l_k + \sum_{t=1}^{T} \delta_{kt})$ based on inequality (24). The upper bound of each part type $u_k = \max\{(p_k^1, 2 \cdot l_k)\}$ and plus an integer generated from $[h_k, \lceil \frac{B}{b_k} \rceil]$. The number of available trucks $G = \min(G_{GM}, G_{LDQ})$, where G_{GM} and G_{LDQ} represent the total number of trucks used in GM and LDQ, respectively. Parameter values for the loss functions in Eqs. (51)–(52) follow the settings in Václavík et al. (2018). Pilot tests using randomly selected instances of various sizes are conducted to define the dataset size as $D = 2T$ per instance, and the tolerance gap $\epsilon = 0.05$ in ET rule 2.

5.2. Results on the performance of BPL algorithm

In this subsection, we evaluate the effectiveness and efficiency of the proposed BPL algorithm through comprehensive computational experiments. The main objectives are: (1) to compare BPL's performance against three benchmark methods—directly solving the integer programming (IP) model using CPLEX, a practical greedy heuristic (GM), and a classic Genetic Algorithm (GA); (2) to assess both solution quality and computational efficiency; and (3) to examine how BPL scales with problem size. These evaluations are structured to highlight BPL's robustness, solution optimality, and runtime performance across problem instances of varying scale.

For each test instance, we solve first the proposed IRP variant using both the GM method (as applied in practice) and the GA algorithm. Let Z_{GM} denote the objective value obtained by the GM method, and Z_{GA} the value produced by the GA algorithm. The corresponding IP model is then solved using CPLEX. Due to the computational complexity, only small-scale instances can be solved to optimality within a reasonable time frame. Therefore, a time limit of 3600 s is set for the first 20 groups of instances, and 7200 s for the largest group. Let Z_{IP} be the best objective value found by CPLEX within the time limit, Z_{BPL} the result from the BPL algorithm, and Z_{GLB} the best lower bound obtained from the BPL algorithm's branching tree.

Table 3 presents the experimental outcomes across 21 instance groups. The first column is the number of the group index, and the next three columns are the configuration of each group, indicating the values of K, N, and T. The column labeled BPL-GLB reports the average relative gap between Z_{BPL} and Z_{GLB} over the instances of each group, which is calculated by $\frac{Z_{BPL}-Z_{GLB}}{Z_{GLB}} \cdot 100\%$. Similarly, BPL-IP reports the average relative gap of Z_{BPL} and Z_{IP}, which is calculated by $\frac{Z_{BPL}-Z_{IP}}{Z_{IP}} \cdot 100\%$; BPL-GM and BPL-GA show the respective average gaps between Z_{BPL} and Z_{GM}, and between Z_{BPL} and Z_{GA}, both expressed in percentage terms. The final three columns provide the average computation times (in seconds) required to obtain Z_{IP}, Z_{BPL}, and Z_{GA}, respectively. Since the goal is to minimize total inventory and transportation costs, a negative gap indicates that BPL outperforms the comparison methods, with smaller gaps being more favorable. The numbers in parentheses under the IP column indicate how many instances in each group could not be solved to optimality by CPLEX within the allotted time. Specifically, only 10 instances in group 1 and 3 in group 2 reached optimality. The computation time for the GM method is omitted, as it typically completes within 1 s.

According to the configuration of each group of instances, we divide them into three different scales: small, medium, and large. The following results can be observed:

Table 4
Experimental results on comparing basic BnP algorithm and the proposed BPL algorithm.

Group	K	N	T	Time (s)		Time	Failure ratio
				BnP	BPL	improvement	
1	5	2	8	26.54	22.63	−14.72%	8.11%
2	5	2	24	102.41	82.52	−19.42%	3.39%
3	5	2	48	162.90	128.38	−21.19%	3.15%
4	10	5	8	26.16	22.89	−12.50%	6.60%
5	10	5	24	146.77	121.80	−17.01%	2.98%
6	10	5	48	262.03	232.68	−11.20%	5.07%
7	10	10	8	60.20	49.42	−17.91%	6.10%
8	10	10	24	146.77	123.42	−15.91%	8.48%
9	10	10	48	279.49	248.94	−10.93%	4.04%
10	15	3	8	128.73	114.00	−11.44%	5.12%
11	15	3	24	213.27	187.17	−12.24%	6.08%
12	15	3	48	469.55	410.25	−12.63%	3.42%
13	15	10	8	122.88	107.52	−12.50%	4.47%
14	15	10	24	151.78	131.00	−13.69%	3.52%
15	15	10	48	631.84	561.58	−11.12%	3.73%
16	20	10	8	146.78	133.25	−9.22%	5.18%
17	20	10	24	239.78	211.80	−11.67%	2.46%
18	20	10	48	576.45	528.60	−8.30%	2.06%
19	20	20	8	223.64	196.71	−12.04%	2.82%
20	20	20	24	579.78	523.72	−9.67%	3.72%
21	20	20	48	786.51	717.06	−8.83%	2.13%
Average				261.16	231.21	−13.05%	4.41%

(i) For small-scale instances (Group1-6), the IP model reaches optimality only for Group 1 and three instances in Group 2. While BPL yields worse objective values than CPLEX's optimal solutions in Group 1, the average gap between Z_{BPL} and Z_{IP} is just 3.37%. Across all six groups, BPL's relative gap with CPLEX averages −7.69%, with BPL significantly faster: average computational times are 3001.67 s for CPLEX versus 101.82 s for BPL.

(ii) For medium-scale instances (Group 7–15), CPLEX fails to reach optimal solution within the given time limit. The average gap between Z_{BPL} and Z_{IP} is −20.73%, showing BPL's solutions are significantly better than CPLEX's best within the time limit. BPL's average computation time is also much shorter at 214.81 s.

(iii) For large-scale instances (Group16-21), BPL outperforms CPLEX on all instances, with an average gap of −29.54% between Z_{BPL} and Z_{IP}. BPL's average runtime is 385.19 s, which is practical. For Group 21, the time limit for CPLEX was extended to 7200 s, as it could not produce feasible solutions within 3600 s for some cases. Even then, BPL shows a significant advantage, with an average gap of −47.14%, underscoring its effectiveness on large-scale instances.

(iv) Across all 21 groups, BPL significantly outperforms the GM and the GA algorithm. The average gap between Z_{BPL} and Z_{GM} is −29.87%, with this gap widening as instance size grows. Although GA is more computationally efficient (average 101.4 s) and shows stable performance, its solutions are consistently worse, with an average gap of −62.49% compared to BPL. BPL consistently produces near-optimal solutions, with a gap of just 4.42% on average from the global lower bound and an average runtime of 231.21 s, demonstrating its superior approximation to optimality over GA.

(v) As the instance scale $(K,\ N,\ T)$ increases, the BPL algorithm maintains stable quality, with computational time increasing but remaining practical. For example, in Groups 7, 13, and 16, as the number of part types K grows while N and T remain constant, the average gap between Z_{BPL} and the global lower bound Z_{GLB} decreases (7.11%, 5.41% and 3.12%, respectively), while computational time rises (49.42 s, 107.52 s, and 133.25 s). A similar trend appears in Group 8, 14, and 17. Additionally, in Groups 4, 5, and 6, with T increasing while K and N stay fixed, the average gaps between Z_{BPL} and Z_{GLB} also decrease slightly (6.35%, 3.46% and 3.56%), with computational time increasing (22.89 s, 121.8 s and 232.68 s). Similar patterns occur when N increases with K and T constant.

5.3. Analysis on the efficiency of BnP

In this subsection, we aim to quantify the contribution of the online regression module within the BPL algorithm. Specifically, we compare the proposed BPL framework, which integrates a machine learning-based regression model, against a baseline Branch-and-Price (BnP) variant that excludes this component based on the instances from Section 5.1. The goal is to evaluate how much the online prediction model accelerates the algorithm and whether it maintains reliable performance.

Table 4 presents the comparison results between the BnP and BPL algorithms across different instance groups. The first four columns replicate the group configurations used in Table 3. The column labeled "Time (sec)" reports the average computation time (in seconds) for each method, with t_{BnP} and t_{BPL} representing the average running times of BnP and BPL, respectively. The column entitled "Time Improvement" shows the proportion of the time reduced, which is calculated by $\frac{t_{BPL}-t_{BnP}}{t_{BnP}} \cdot 100\%$. Column "Failure Ratio" shows the percentage of subproblems that fail to predict a proper upper bound in the entire BPL procedure.

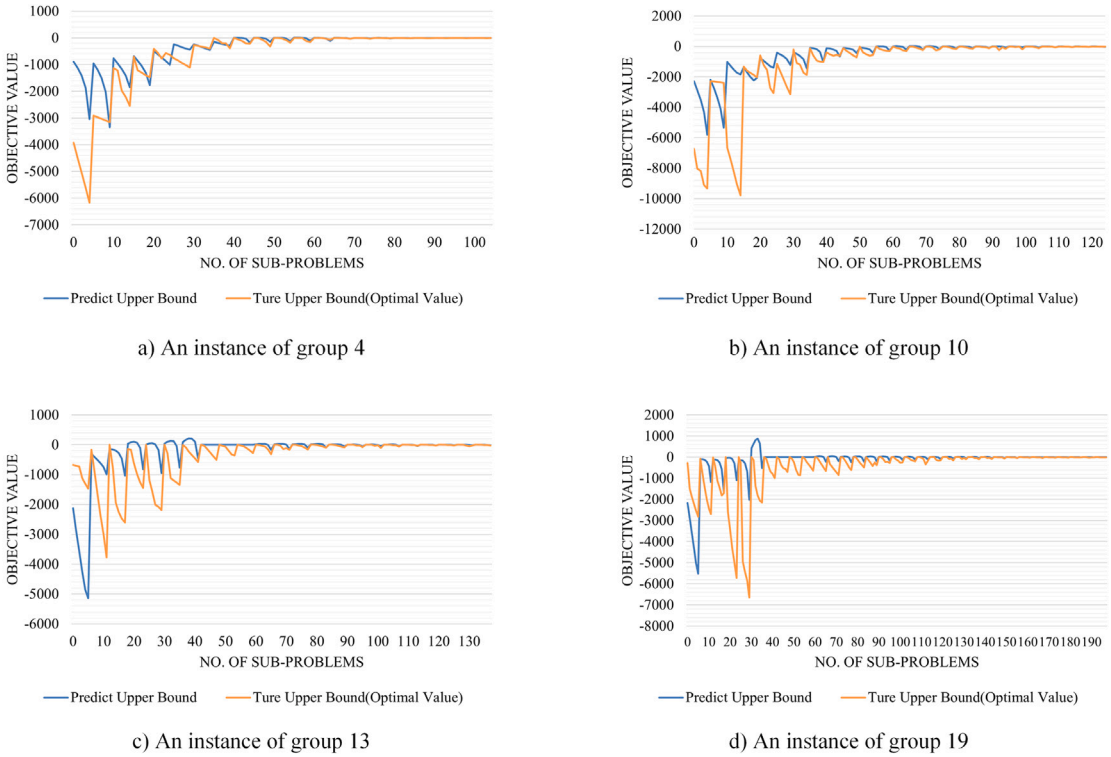


Fig. 5. Comparison between the predicted upper bound and the real upper bound.

Table 5
Different feature vectors.

FV	Description
FV1	$\phi = [\phi_1, \phi_2, \phi_3, f]$ as described in Section 4.3.1
FV2	$\phi = [\phi_1, \phi_2, \phi_3]$, where ϕ_1, ϕ_2, ϕ_3 are the same as FV1
FV3	$\phi = [\pi^1_{kt}, \pi^2_{kt}, \psi_t, f]$, where $\pi^1_{kt}, \pi^2_{kt}, \psi_t$ are the dual variable values
FV4	$\phi = [\phi_1, \phi_2, \phi_3, f]$, where $\phi_1 = \sum_{t=f}^{T} \sum_{k=1}^{K} \pi^1_{kt}$, $\phi_2 = \sum_{t=f}^{T} \sum_{k=1}^{K} \pi^2_{kt}$ and $\phi_3 = \sum_{t=f}^{T} \psi_t$
FV5	$\phi = [\phi_1, \phi_2, \phi_3, f]$, where $\phi_1 = \frac{\sum_{t=f}^{T} \sum_{k=1}^{K} \pi^1_{kt}}{K \cdot (T-f+1)}$, $\phi_2 = \frac{\sum_{t=f}^{T} \sum_{k=1}^{K} \pi^2_{kt}}{K \cdot (T-f+1)}$, ϕ_3 is same as FV1
FV6	$\phi = [\phi_4, f]$, where $\phi_4 = \frac{\phi_1+\phi_2+\phi_3}{3}$ and ϕ_1, ϕ_2, ϕ_3 are same as FV1
FV7	$\phi = [\phi_1, \phi_2, \phi_3, \pi^1_{kt}, \pi^2_{kt}, \psi_t, f]$, i.e., FV1+ FV3

The results in Table 4 indicate that the online regression technique embedded in the proposed BPL algorithm performs effectively and significantly accelerates the solution process across different instance sizes. The average failure rate over the 21 groups of instances is 4.41%, and the average failure rate of each group is lower than 8.5%, which indicates that the online regression model outputs good predictions in most cases. Compared to solving the problem by the original BnP algorithm, applying the online regression to predict cuts for the subproblems saves 13.05% of the computing time on average over the 21 groups of instances.

Fig. 5 illustrates the predicted upper bound values alongside the real upper bounds for all subproblems at the root nodes of four instances as examples. The figures show that, following several early iterations of the CG algorithm, the online regression is able to produce fairly accurate and tight upper-bound estimates for the subproblems.

5.4. Analysis on the feature vector

This subsection investigates the impact of feature vector design on the performance of the online regression module embedded in the BPL algorithm. The goal is to evaluate how different representations of subproblem characteristics affect both prediction accuracy and computational efficiency. To this end, we compare the baseline feature vector FV1, used in our BPL implementation (as defined in Section 4.3.1), with six alternative feature vectors that emphasize different problem structures. Table 5 describes these variants.

A set of groups is selected from the instances described in Section 5.2, representing different instance scales to test the seven feature vectors. Since different eigenvectors only affect the performance of the online regression model and the computational efficiency rather than the results on solving the subproblems, we compare the BPL algorithm's computation time and the percentage

Table 6
Experimental results based on different feature vectors.

Group	FV1		FV2		FV3		FV4		FV5		FV6		FV7	
	CT (s)	FR (%)	CT (s)	FR (%)	CT (s)	FR (%)	CT (s)	FR (%)	CT (s)	FR (%)	CT (s)	FR (%)	CT (s)	FR (%)
2	41.58	6.21	40.3	2.73	51.18	45.91	42.93	17.88	40.26	6.21	42.2	5.23	49.7	41.80
5	134.05	3.26	134.46	6.74	174.85	44.44	159.88	22.40	151.72	28.03	140.36	27.50	142.55	33.33
8	81.33	9.63	85.61	15.72	96.05	40.00	88.13	19.70	87.89	16.60	86.29	16.97	105.58	26.49
13	66.76	3.64	64.29	1.21	73.35	45.45	66.11	12.12	67.07	4.24	65.53	5.45	74.68	48.59
16	182.43	4.70	169.75	0.00	199.83	25.64	186.4	31.62	172.99	3.85	188.75	4.70	278.72	42.08
19	127.63	4.90	193.16	9.80	203.41	36.07	207.53	8.50	200.86	2.50	197.51	1.07	180.6	28.19
Ave.	105.63	5.39	114.60	6.03	133.11	39.59	125.16	18.70	120.13	10.24	120.11	10.15	138.64	36.75

of failures of the prediction while applying different feature vectors. Table 6 presents these results, with the first column listing the selected instance group indices, followed by columns displaying outcomes for BPL using feature vectors FV1 through FV7. Under each feature vector, two metrics are reported: average computation time (CT) in seconds and average failure ratio (FR). The final row labeled "Ave." summarizes the average results across all selected groups.

According to the results in Table 6, we have the following observations: (i) Among the seven feature vectors tested, the feature vector FV1, which is the one selected for the proposed BPL algorithm, performed best in terms of calculation time and prediction success rate. The average calculation time of the BPL algorithm applying FV1 is 105.63 s over the tested groups, which is the shortest among the tested feature vectors, and the average prediction failure ratio is 5.39%, which is the smallest among the tests. (ii) When more features are considered in the vector, it results in a longer computational time and a higher failure rate. For example, FV3 and FV7 consider all dual variables of the RRMP. The BPL algorithms applying FV3 and FV7 cost an average calculation time equal to 133.11 s and 138.64 s for solving the tested instances, respectively, which increase by 26% and 31% compared to that of FV1. For the prediction failure ratio, the application of FV3 and FV7 to the BPL resulted in the ratio growing sharply to 39% and 37%, respectively, which was an increase of 7.3 times and 6.8 times compared with that of applying FV1, respectively. Based on the above observations, it implies that the efficiency and accuracy of BPL can be improved by choosing the best feature set, and it is important to find the trade-off between efficiency and complexity, since higher feature count may increase computational time and prediction failure rate.

5.5. Analysis on different branching strategies

This subsection evaluates the effectiveness of our hybrid branching strategy (BS1) within the BPL framework. The goal is to determine whether combining multiple branching rules (as done in BS1) offers tangible benefits over standard, single-rule branching approaches commonly used in BnP algorithms. We benchmark the performance of BS1 against three alternative branching strategies, each representing a typical choice in the literature or a component of BS1:

- BS1: The hybrid branching strategy proposed in our algorithm.
- BS2: Branching Rule 1 of the hybrid branching strategy BS1.
- BS3: Branching Rule 2 of the hybrid branching strategy BS1.
- BS4: A single-variable most-infeasible branching strategy which selects the variable x_{h^*} whose value is closest to 0.5, and then branches into $x_{h^*} = 0$ and $x_{h^*} = 1$.

The proposed algorithm is tested with each branching strategy based on one instance randomly selected from groups 2, 5, 8, 13, 16, and 19, respectively. Fig. 6 displays the results. The bars show the average gap between the algorithm output (Z_{BPL}) and the global lower bound (GLB) across the six instances for each strategy, while the line indicates the computation time (in seconds) for each strategy.

The test results in Fig. 6 show that our proposed hybrid branching strategy (BS1) outperforms the other three strategies. In terms of the gap between Z_{BPL} and GLB, BS1 attains an average gap of 7.57%, outperforming the other strategies which have gaps of 8.57%, 7.97%, and 9.03%. Regarding computational time, BS1 requires an average of only 64.27 s, outperforming the other strategies, which take 70.96 s, 92.27 s, and 64.86 s, respectively. This indicates that BS1 reduces computational time by 10%–43% compared to using the single rules in BS2 and BS3, while providing better solutions in a similar timeframe to the commonly used simple rule in BS4. These results underscore the effectiveness of our proposed hybrid branching strategy.

5.6. Analysis on the cost coefficients and the number of trucks

In this subsection, a sensitivity analysis is performed to investigate the impact of variations in three critical parameters on the performance of the BPL algorithm: (1) the unit routing cost C_1; (2) the unit inventory cost C_2; and (3) the number of trucks G. These are the main tunable parameters of the inventory routing problem (IRP), apart from the structural dimensions K (part types), N (suppliers), and T (planning horizon), which were already analyzed in Section 5.2.

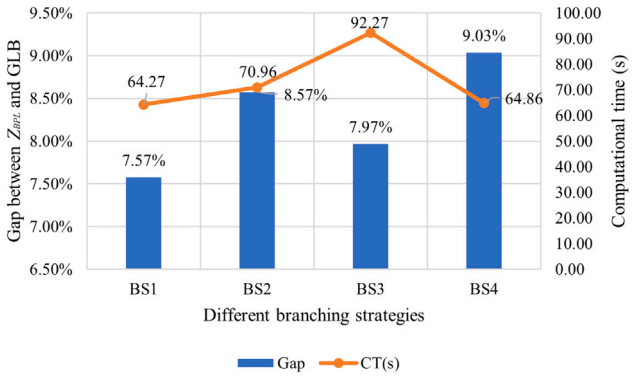


Fig. 6. Performance of BPL using different branching strategies.

Table 7
Experimental results under different values of inventory cost coefficient C_1.

C_1	0.010	0.035	0.050	0.075	0.100	0.125	0.150	0.175	0.200	0.225	0.250	0.275	0.300
IC	942	3241	5619	6826	9152	10 950	12 379	13 667	15 206	18 697	22 343	24 735	26 709
RC	4416	4526	4434	4609	4879	4776	4760	4808	4847	4799	4866	4829	4892
TC	5358	7766	10 052	11 469	14 029	15 725	17 139	18 475	20 053	23 496	27 376	29 397	31 599

Table 8
Experimental results under different values of transportation cost coefficient C_2.

C_2	20	40	60	80	100	120	140	160	180	200
IC	9129	9122	9759	11 226	11 285	11 345	11 341	11 321	11 351	12 850
RC	1056	2179	2944	3873	4751	6092	6609	7857	8078	8157
TC	10 185	11 301	12 703	15 099	16 036	17 438	17 951	19 178	19 429	21 006

We use representative instances from Groups 2, 5, 8, 13, 16, and 19, corresponding to small-, medium-, and large-scale scenarios, as test cases. For each parameter, we analyze its impact on three outcomes: total cost (TC), inventory cost (IC), and traveling cost (RC).

First, for cost coefficient of inventory, C_1, we test the selected instances with C_1 = 0.010, 0.035, 0.050, 0.075, 0.100, 0.125, 0.150, 0.175, 0.200, 0.225, 0.250, 0.300. Note that the values of C_1 are determined by the real values of the objective's coefficients in practice. For different values of C_1, the selected instances are solved by BPL algorithm, and the results of the average outcomes, IC, RC, and TC over the selected groups are shown in Table 7.

Accordingly, Fig. 7 illustrates the changes in the average inventory cost, routing cost and total cost over the selected test groups under different values of C_1. It can be observed that, as the unit inventory cost coefficient C_1 relatively uniformly increases, the total inventory cost in the proposed IRP problem also increases in a uniform way, and the total cost increases simultaneously at a similar rate. For the average routing cost, it changes sharply in a growing pattern when the value of C_1 increases from 0.010 to 0.100, and then stays relatively stable when the value of C_1 increases from 0.125 to 0.300. For example, when the unit cost C_1 changes from 0.050 to 0.100, the routing cost rises from 4434 to 4879, with an increasing ratio of 10.41%; when it increases from 0.100 to 0.200, the average routing cost decreases slightly before increasing, but the value stays between 4760 and 4879. The results indicate that, for the part types with a small unit inventory cost, the IRP optimization decisions have a greater impact on the transportation cost than on the inventory cost, while for the part types with a larger unit inventory cost, the IRP optimization decisions have less impact on the transportation cost. The automotive manufacturer could pay more attention to the collection route design for the part types with small unit inventory costs.

For the cost coefficient on transportation, C_2, we test the selected instances with C_2 = 20, 40, 60, 80, 100, 120, 140, 160, 180 and 200. Similar to C_1, the values of C_2 are determined based on the real values of the parameters in practice, and the selected instances are solved by the BPL algorithm under the different values of C_2. Table 8 reports the results.

Fig. 8 illustrates the change of average inventory cost, routing cost, and total cost over the tested groups with the increase of the unit cost on transportation, C_2. As the value of C_2 increases, the routing cost increases more sharply than the total cost, since the inventory cost stays relatively stable. When the unit transportation cost coefficient increases from 20 to 200, the average routing cost over the tested instances increases from 1056 to 8157, which is more than 7 times, while the average inventory cost increases only slightly from 9129 to 12 850. The results indicate that the unit transportation cost influences the total cost a lot, and the automotive manufacturers should pay more attention to the routing decisions when the features related to unit transportation cost, such as fuel price and labor cost, change in practice.

Finally, we analyze the change in total cost when the number of available trucks, G, increases from 4 to 25. The results are shown in Fig. 9. Note that some of the instances could be infeasible if G is too small, and thus four of the lines in Fig. 8 do not start

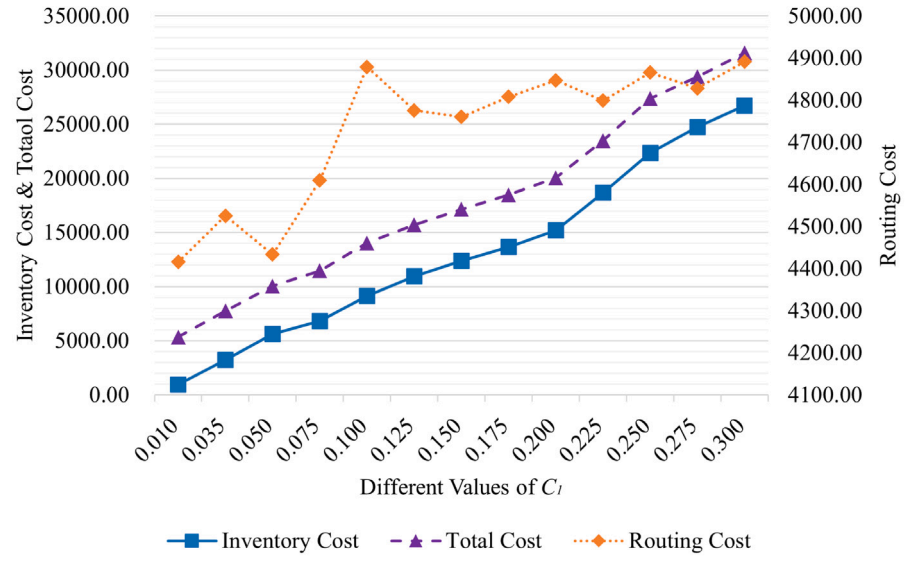


Fig. 7. Change of the costs under different C_1 values.

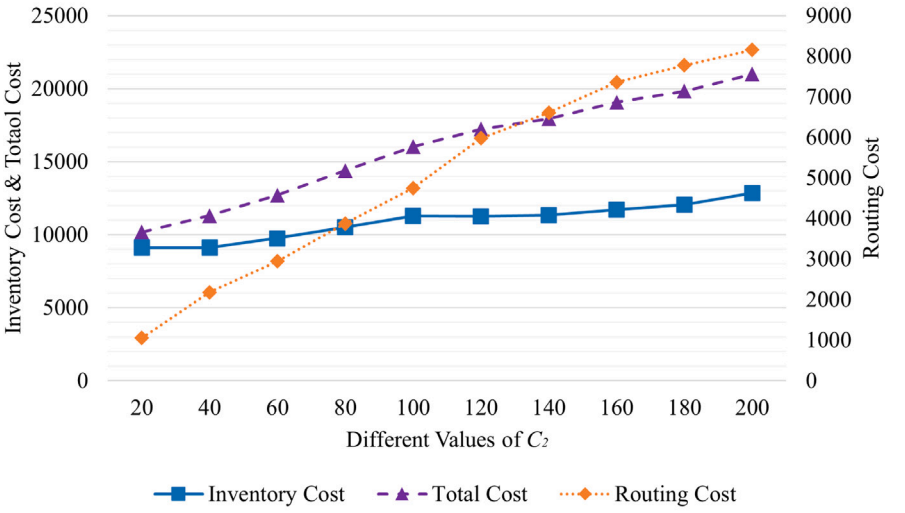


Fig. 8. Change of the costs under different C_2 values.

from $G = 4$. It can be seen from the figure that, for group 8 and group 19, the value of part types K is equal to that of suppliers N, respectively, and the total cost stays almost stable when more available trucks are considered. For group 5 and group 13, where $K/N = 2$ and 1.5, respectively, the average total costs reduced slightly at first before being relatively stable with the value of G getting larger. For group 2 and group 16, where $K/N = 2.5$ and 2 and the proportions are larger than those of groups 5 and 13, the average total costs reduced sharply at the beginning when the value of G increases and stayed relatively stable afterward. The results indicate that the automotive manufacturer could ask the suppliers to be gathered, such as in a supplier park, which is equivalent to multiple part types being supplied at a smaller number of locations, i.e. a larger K/N proportion. In this case, increasing a certain number of trucks could effectively reduce the total inventory and routing cost, but it is not always the more the better.

6. Conclusions and future directions

To address the need for flexible and cost-effective logistics in the increasingly customized automotive industry, this paper considers a multi-period IRP variant for parts inbound logistics under a demand-pull manufacturing mode. The core challenge is to minimize both inventory and routing costs for a fleet of capacitated trucks delivering parts to an automaker, while accounting for

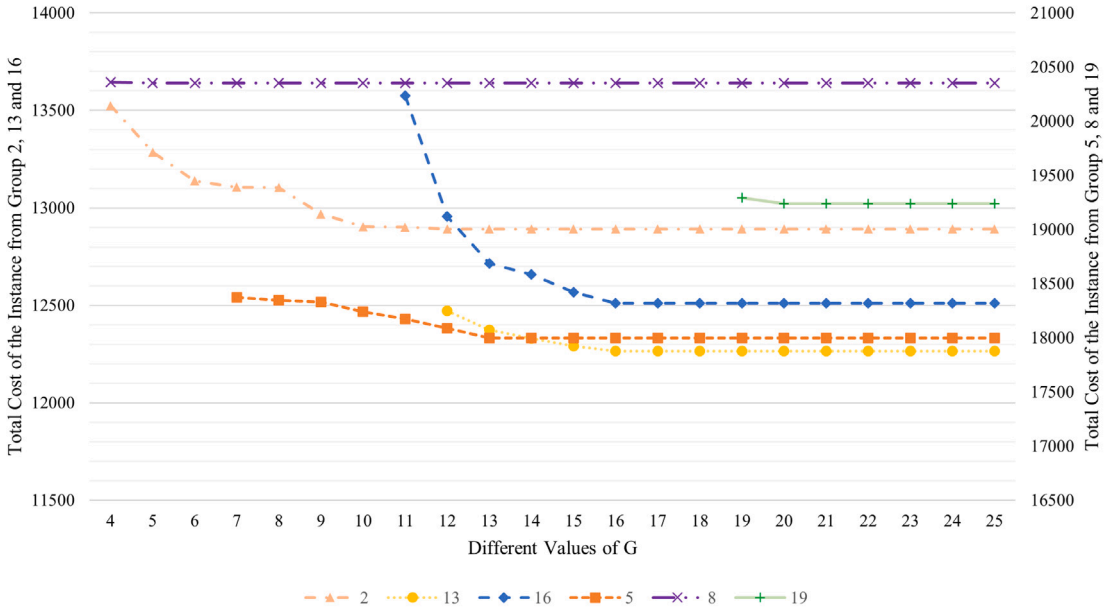


Fig. 9. Change of total cost with different number of available trucks G.

time-varying demands on the assembly line and the predetermined packaging constraints from suppliers. Our problem is initially modeled as an integer program with time-indexed variables and then reformulated based on Dantzig–Wolfe decomposition to enhance tractability. To solve this complex problem efficiently, we introduce a BnP algorithm enhanced with machine learning (BPL). This approach integrates an online regression method to predict upper bounds for the subproblems, providing cuts that accelerate solutions in the column generation process. Hybrid branching strategies and early termination rules are also designed to speed up the BPL approach. Experiments on realistic instance scales show that the proposed BPL approach significantly outperforms traditional CPLEX IP solving, a genetic algorithm (GA), and the greedy method currently used in practice. Our acceleration techniques, including tailored branching rules and machine learning enhancements in column generation, reduce computation time considerably. The proposed BPL algorithm is practical for real-world logistics operations in the automotive industry and other mixed-model assembly plants, offering substantial cost savings and improved decision-making efficiency. Furthermore, the acceleration strategies developed here can be generalized to similar scheduling optimization problems.

This paper focuses on optimizing the inventory routing decisions for auto parts based on the given production sequence in a large-scale mixed-model assembly line, and accelerating the solving of the subproblems in CG-based algorithms. Therefore, future research can be carried out from the following aspects: (1) Considering the collaborative optimization of production sequencing and inbound logistics to further improve customer satisfaction and reduce the total logistics cost; (2) Developing parallel programming methods and other techniques to accelerate further the branch-and-price procedure, such as applying online machine learning to predict global upper/lower bounds to design better branching and bounding strategies and accelerating the solving process of sub-nodes using the predictions.

CRediT authorship contribution statement

Yu Wang: Writing – review & editing, Writing – original draft, Validation, Resources, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Renrong Zheng:** Writing – original draft, Visualization, Validation, Methodology, Formal analysis, Data curation. **Chengji Liang:** Validation, Resources, Methodology, Investigation. **Jian Shi:** Writing – review & editing, Visualization, Validation, Resources.

Declaration of Generative AI and AI-assisted technologies in the writing process

The declaration does not apply to this work. Only grammar-checking tools are used during the preparation of this work to check typos and improve the readability and language of the manuscript.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Yu WANG reports financial support was provided by Science and Technology Commission of Shanghai Municipality. Chengji Liang reports financial support was provided by National Natural Science Foundation of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was funded by Shanghai Sailing Program (21YF1416400) and National Natural Science Foundation of China (71972128).

References

Archetti, C., Ljubić, I., 2022. Comparison of formulations for the inventory routing problem. European J. Oper. Res. 303, 997–1008.
Archetti, C., Speranza, M.G., 2016. The inventory routing problem: the value of integration. Int. Trans. Oper. Res. 23, 393–407.
Baals, J., Emde, S., Turkensteen, M., 2023. Minimizing earliness-tardiness costs in supplier networks—A just-in-time truck routing problem. European J. Oper. Res. 306 (2), 707–741.
Baller, R., Fontaine, P., Minner, S., Lai, Z., 2022. Optimizing automotive inbound logistics: A mixed-integer linear programming approach. Transp. Res. Part E: Logist. Transp. Rev. 163, 102734.
Bell, W.J., Dalberto, L.M., Fisher, M.L., Greenfield, A.J., Jaikumar, R., Kedia, P., Mack, R.G., Prutzman, P.J., 1983. Improving the distribution of industrial gases with an on-line computerized routing and scheduling optimizer. Interfaces 13 (6), 4–23.
Boysen, N., Emde, S., Hoeck, M., Kauderer, M., 2015. Part logistics in the automotive industry: Decision problems, literature review and research agenda. European J. Oper. Res. 242 (1), 107–120.
Cavero, S., Laguna, M., Pardo, E.G., 2024. Solving a short sea inventory routing problem in the oil industry. Comput. Ind. Eng. 189, 109978.
Chen, J., Wang, K., Huang, Y., 2021. An integrated inbound logistics mode with intelligent scheduling of milk-run collection, drop and pull delivery and LNG vehicles. J. Intell. Manuf. 32, 2257–2265.
Chitsaz, M., Cordeau, J.-F., Jans, R., 2020. A branch-and-cut algorithm for an assembly routing problem. European J. Oper. Res. 282 (3), 896–910.
Chuah, K.H., Yingling, J.C., 2005. Routing for a just-in-time supply pickup and delivery system. Transp. Sci. 39 (3), 328–339.
Coelho, L.C., Cordeau, J.-F., Laporte, G., 2014. Thirty years of inventory routing. Transp. Sci. 48 (1), 1–19.
Coelho, L.C., Maio, A.D., Laganà, D., 2020. A variable MIP neighborhood descent for the multi-attribute inventory routing problem. Transp. Res. Part E: Logist. Transp. Rev. 144, 102137.
Desaulniers, G., Rakke, J.G., Coelho, L.C., 2016. A branch-price-and-cut algorithm for the inventory-routing problem. Transp. Sci. 50, 1060–1076.
Diabat, A., Bianchessi, N., Archetti, C., 2024. On the zero-inventory-ordering policy in the inventory routing problem. European J. Oper. Res. 312 (3), 1024–1038.
Duguay, C.R., Landry, S., Pasin, F., 1997. From mass production to flexible/agile production. Int. J. Oper. Prod. Manage. 17 (12), 1183–1195.
Harrison, A., 2001. Perestroika in automotive inbound. Manuf. Eng. 80 (6), 247–251.
Hosseini, S.D., Shirazi, M.A., Ghomi, S.M.T.F., 2014. Harmony search optimization algorithm for a novel transportation problem in a consolidation network. Eng. Optim. 46 (11), 1538–1552.
Jia, M., Chen, F., 2023. Upward scalable vehicle routing problem of automobile inbound logistics with pickup flexibility. Transp. Res. Part E: Logist. Transp. Rev. 177, 103253.
Jiang, Z., Huang, Y., Wang, J., 2010. Routing for the milk-run pickup system in automobile parts supply. In: Proceedings of the 6th CIRP-Sponsored International Conference on Digital Enterprise Technology. Springer, pp. 1267–1275.
Kheiri, A., 2020. Heuristic sequence selection for inventory routing problem. Transp. Sci. 54, 302–312.
Konstantakopoulos, G.D., Gayialis, S.P., Kechagias, E.P., 2022. Vehicle routing problem and related algorithms for logistics distribution: A literature review and classification. Oper. Res. 22 (3), 2033–2062.
Lin, Y., Xu, T., Bian, Z., et al., 2015. A two-phase heuristic algorithm for the common frequency routing problem with vehicle type choice in the milk run. Math. Probl. Eng. 2015.
Liu, A., Zhu, Q., Xu, L., Lu, Q., Fan, Y., 2021. Sustainable supply chain management for perishable products in emerging markets: An integrated location-inventory-routing model. Transp. Res. Part E: Logist. Transp. Rev. 150, 102319.
Mahjoob, M., Fazeli, S.S., Tavassoli, L.S., Mirmozaffari, M., Milanlouei, S., 2021. A green multi-period inventory routing problem with pickup and split delivery: A case study in flour industry. Sustain. Oper. Comput. 2, 64–70.
Mahmutoğulları, Ö., Yaman, H., 2023. A branch-and-cut algorithm for the inventory routing problem with product substitution. Omega 115, 102752.
Mao, Z., Huang, D., Fang, K., Wang, C., Lu, D., 2020. Milk-run routing problem with progress-lane in the collection of automobile parts. Ann. Oper. Res. 291, 657–684.
Morabit, M., Desaulniers, G., Lodi, A., 2021. Machine-learning–based column selection for column generation. Transp. Sci. 55 (4), 815–831.
Oğuz, O., 2002. Generalized column generation for linear programming. Manag. Sci. 48 (3), 444–452.
Ohlmann, J.W., Fry, M.J., Thomas, B.W., 2008. Route design for lean production systems. Transp. Sci. 42 (3), 352–370.
Rau, H., Budiman, S.D., Widyadana, G.A., 2018. Optimization of the multi-objective green cyclical inventory routing problem using discrete multi-swarm PSO method. Transp. Res. Part E: Logist. Transp. Rev. 120, 51–75.
Skálnes, J., Andersson, H., Desaulniers, G., Stålhane, M., 2022. An improved formulation for the inventory routing problem with time-varying demands. European J. Oper. Res. 302 (3), 1189–1201.
Tiniç, G.Ö., Koca, E., Yaman, H., 2021. An exact solution approach for the inventory routing problem with time windows. Comput. Oper. Res. 134, 105371.
Václavík, J., Novák, T., Šücha, P., Hanzálek, Z., 2018. Accelerating the branch-and-price algorithm using machine learning. European J. Oper. Res. 271 (3), 1055–1069.
Wang, Y., Chen, F., 2019. Packed parts delivery problem of automotive inbound logistics with a supplier park. Comput. Oper. Res. 101, 116–129.
Yang, Y., Boland, N., Dilkina, B., Savelsbergh, M., 2022. Learning generalized strong branching for set covering, set packing, and 0–1 knapsack problems. European J. Oper. Res. 301 (3), 828–840.
Yu, Y., Wang, S., Wang, J., Huang, M., 2019. A branch-and-price algorithm for the heterogeneous fleet green vehicle routing problem with time windows. Transp. Res. Part B: Methodol. 122, 511–527.
Zajac, S., Huber, S., 2021. Objectives and methods in multi-objective routing problems: a survey and classification scheme. European J. Oper. Res. 290, 1–25.